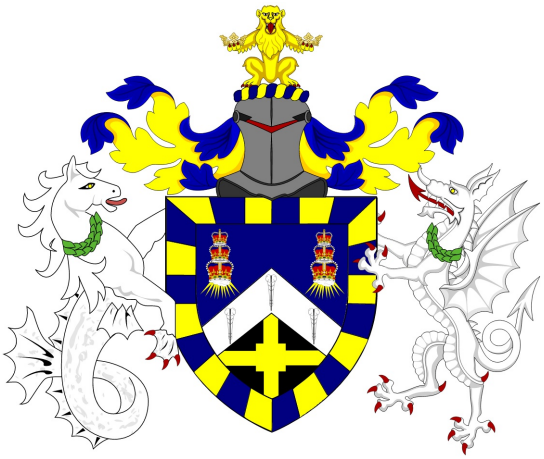
Data Analytics MSc Dissertation MTHM038, 2023/24

Using Machine Learning to Predict football matches

Applied to the 2024/25 Premier League Season

George Francis Massey, ID 230973275

Supervisor: Dr. Subhajit Jana



A thesis presented for the degree of
Master of Science in *Data Analytics*

School of Mathematical Sciences
Queen Mary University of London

Declaration of original work

This declaration is made on September 2, 2024.

Student's Declaration: I George Massey hereby declare that the work in this thesis is my original work. I have not copied from any other students' work, work of mine submitted elsewhere, or from any other sources except where due reference or acknowledgement is made explicitly in the text. Furthermore, no part of this dissertation has been written for me by another person, by generative artificial intelligence (AI), or by AI-assisted technologies.

Referenced text has been flagged by:

1. Using italic fonts, **and**
2. using quotation marks "...", **and**
3. explicitly mentioning the source in the text.

Abstract

In the last few decades the use of data in sport has revolutionised the industry. This started in American sports, particularly baseball, where it was first shown how data-driven teams can outperform others. The data wave has made its way into one of the most popular sports in the world, football. Due to its popularity and the adoption of data, football match and season predictions have become of great interest to fans, media and the sport betting industry. There have been many studies that have proposed different ways in predicting football matches. Main approaches consist of either predicting the result (home win/draw/away win) or the score, i.e. number of home and away goals. This paper investigates the latter by building multiple machine learning models to predict the number of goals scored by each team in a football match. From this we can indirectly predict the result of the game and then simulate all matches from a football season. These models were trained on Premier League matches from 2017 to 2023 and tested on the 2023/2024 season. The selected model was then used to forecast the upcoming 2024/2025 season.

Contents

1 Introduction **5**
- 1.1 Problem Statement . . . 5
- 1.2 Approach . . . 6

2 Data **7**
- 2.1 Source . . . 7
- 2.2 Preprocessing . . . 8
- 2.3 Variable Correlation Analysis . . . 11
- 2.4 Target Variables and Home-Field Advantage . . . 12
- 2.5 Poisson Distribution . . . 12
- 2.6 Pearson's Chi-square Goodness of Fit Test . . . 14

3 Modelling Methods **17**
- 3.1 Poisson Regression . . . 17
  - 3.1.1 Generalised Linear Models . . . 17
  - 3.1.2 Link Function . . . 17
  - 3.1.3 Likelihood Function . . . 18
  - 3.1.4 Independence . . . 19
  - 3.1.5 Hyper Parameters . . . 20
  - 3.1.6 Tuning and Grid Search . . . 22
  - 3.1.7 Monte Carlo Simulations . . . 23
  - 3.1.8 Results . . . 26
- 3.2 Logistic Regression . . . 26
  - 3.2.1 Binary Logistic Regression . . . 26
  - 3.2.2 Multinomial Logistic Regression . . . 29
  - 3.2.3 Likelihood Function . . . 31
  - 3.2.4 Multi output Classification . . . 31

3.2.5 Hyper Parameter Tuning 32
3.2.6 Results 32

4 Model Selection **35**
4.1 Model Comparison 35
4.1.1 Model Discussion 35
4.1.2 Repeated K-Fold Cross Validation 36
4.2 Feature Analysis - Bootstrap 36

5 Forecast **39**
5.1 Approach 39
5.2 Results 40

6 Conclusion **43**

A Source Code **45**
A.1 Implementation of `def scrape_matches` 45
A.2 Implementation of `def get_market_value` 46
A.3 Implementation of `def get_weighting` 47
A.4 Implementation of `def rolling averages` for General Form Variables 47
A.5 Implementation of `def rolling averages` for Home and Away form Variables 47
A.6 Implementation of `def calculate_goal_probabilities` . . . 48
A.7 Implementation of `def simulate_game` 48
A.8 Implementation of `def simulate_all_games` 48
A.9 Implementation of `def simulate_all_future_games` 50
A.10 Implementation of `def bootstrap` 51
A.11 Implementation of `def grid_search` 52
A.12 Implementation of `def grid_search_log_reg` 52
A.13 Implementation of `def kfold_cv` 53

B References **54**

Chapter 1

Introduction

1.1 Problem Statement

Football enthusiasts often discuss where they think each team will finish in the upcoming season. The main problem being that fans are generally over optimistic about where they think their team will finish. However, what if we could build an unbiased model. For example, team X, has a 33% probability to win the league this season. Often papers addressing this problem have an angle of trying to beat the bookies, e.g. Bruinsma (2020) exploited inefficiencies in the betting market based on his model's predictions with some relative success [3]. However, this paper will not explore this process. To attempt to consistently beat betting companies who are constantly developing and improving their own advanced machine learning models to obtain these odds, would likely be a fruitless exercise. The reason for investigating this problem, as a massive football fan, is out of pure intrigue into the idea of attempting to predict football matches.

The Premier League is the top football division in England and arguably the world as it has the highest league ranking (Transfermarkt, 2024a) and the highest overall market value of players (Transfermarkt, 2024b) [15] [16]. It is also the most popular league in the world with each match being broadcast in up to 643 million homes, so trying to predict outcomes is a hot topic. Furthermore, the low-scoring nature of football compared to sports like basketball makes it more difficult to predict.

1.2 Approach

When attempting to predict the number of goals each team will score in a football match, most literature keeps it simple by calculating an attack rating and a defence rating for each team based on their own metrics. For example, Zerbari *et al.* (2021) calculated ratings based on the average goals scored/conceded by each team in the previous season compared to the league average [33]. Whilst this is easily computed it removes one key variable of predicting football matches, the form of each team. Good form, in general, as defined by the Cambridge dictionary is "to be performing well" [4]. This translates to a phenomenon in sport that causes teams to go on 'hot' streaks, winning multiple matches in a row due to the increased confidence from performing well. Similarly the opposite, poor form, can happen when a team has lost a few matches on the bounce. Therefore our data has been split into different form based metrics whilst also using other longer term performance metrics.

Our approach to predicting the Premier League season will be from predicting the score of each game and then calculating the subsequent points each team achieves. Due to football's low scoring nature, there is often an element of luck or randomness in the scores which makes the exact score very difficult to predict. Therefore, most literature, like Saraiva *et al.* (2016) tends to focus on predicting the probability a team scores a certain number of goals [17]. To help explain why, here's an example. Effective models and bookmakers, if they were forced to give a straight answer, would predict Manchester City to win each game they play in the league, as they are the best team in the league and have won the league for the last four seasons. However, to suggest any team would actually win all 38 games would be an extremely unlikely prediction as it would be virtually impossible for them to not slip up in at least one game. This increases the sports popularity as the favourite can sometimes lose. To account for this, and extend previous research, Zerbari *et al.* (2021) [33], that focuses on individual game predictions we use a Monte Carlo method to simulate games using our goal prediction probabilities. When simulating whole seasons numerous times this will attempt to capture all those occasions where the underdog takes down the favourite, giving a more sound league table prediction.

Chapter 2

Data

2.1 Source

The data used in this paper was from Fbref.com. Appendix A.1 shows the `scrape_matches` function built in Python to access this data. This first scraped the final league table data [6] and then each teams individual match data [7] from the season by accessing their provided URL. The inputs needed were the original league table URL, the league name and year range of the data we wanted to obtain which in this instance was from 2017-2023. League name was specified as we needed to scrape data from the Championship for teams that were promoted to the Premier League. Each row of the returned dataframe corresponded to a team and their opponent for that match. Therefore each game appears twice in the dataset, once from the perspective of both teams in the game. Data from the match (columns of the dataframe) included a variety of statistics, from attendance numbers to goals scored. These columns were stripped to focus just on specific items of interest for our model.

The choice of including only data from 2017 onwards was due to this being when expected data was introduced by Fbref.com. Expected data in football is a relatively new measure which in its simplest form is called expected goals (xG). The statistic xG is the probability that a shot will result in a goal based on a variety of factors, i.e. location of shot, keeper position, defender position etc [8]. Expected goals has become popular in assessing

the underlying performance of teams. For example a team may score 1 goal in a game but create an xG of 3 which would suggest they were unlucky to only score 1 goal and actually played better than their one goal suggests. Factoring in the expected data that a team creates and concedes in the game will modernise similar papers, like (Saraiva, 2016 [17]), surrounding predicting football matches due to their age.

2.2 Preprocessing

Each row in the desired dataset represents a unique premier league match, with the columns representing different statistics about the home and away team. The reason for separating the teams into home and away instead of team 1 and team 2 was because of something called the home-field advantage, refer to section 2.4 for further discussion. Features (columns) include form-based data for each team and other historical, longer term performance metrics. The model makes a score prediction for each match, in the form of how likely each team was to score a certain number of goals given each teams data.

A quite extensive preprocessing task started by dealing with one of the most difficult factors of this problem. Each season the top three teams are promoted from the Championship to the Premier League. They replace the bottom three, relegated teams from the Premier League, meaning the teams in the league change each season. This caused a problem in trying to measure past performances of teams who were playing in different leagues, which vary in difficulty. Appendix A.3 shows the proposed solution by creating a weighting system function to even the playing field for all teams. The function `get_weighting` compares the average of each metric of the promoted and relegated teams. For example, in our data, relegated Premier League teams, on average, achieve 27.5 points compared to 91.4 for promoted Championship teams. All data was adjusted, i.e. goals scored, promoted teams on average score 0.75 more goals per game then the relegated Premier League sides. The adjustment puts the promoted teams at a similar level to the bottom three in the Premier League at the start of the season. Andrews (2019) used a different approach by assigning all promoted teams the same arbitrary rating [1], however, this fails to differentiate between the three teams. For example, the team that wins the Championship could finish 20 to 30 points above the

playoff winner, which should be not be ignored.

Predictor variables have been split into two groups to allow for easier reading and to reference variables more easily. The first group were form-based variables. There were 3 variables in this group, however as there was one for each team (home and away), technically 6. The first variable refers to each team's general form over the last 5 games they have played, calculated in Appendix A.4 using the `rolling_averages` function. This included both home and away games going back 5 games, to capture their immediate form going into the match. It was calculated by taking a rolling average of 50% of their goal difference added to 50% of their expected goal difference over the last 5 games. This should capture not just their observed form but also their underlying form data. This variable was labelled AdjGD form 5.

The other two variables in this group were similar but instead account for how well the home team performed at home and how well the away team performed away. This allows the model to capture the difference in how teams perform dependent on the venue, therefore accounting for home-field advantage type effects. Appendix A.5 shows how the previous `rolling_averages` function has been used to calculate the average adjusted goals scored and conceded over the last 10 home/away games. Adjusted goals were calculated by adding together 50% of goals scored/conceded and 50% of expected goals scored/conceded. These variables have been labelled Home/Away GS/GC form 10. These variables allow the model to account, for example, if the home team has a high goal scoring form and the away team has a high goal conceding form, that we may expect the home team to score a high number of goals.

The second group of predictor variables relates to the long term performance and skill of each team. The first variable was the market value of each team at the start of the season, to capture the skill of the players in each team. The function `get_market_value`, shown in Appendix A.2, scraped yearly market value data from Transfermarkt [16], and assigned each value to the corresponding team and season. Transfermarkt is the most commonly used, industry standard, website for player market values. The final variable used in our analysis was the number of points each team finished on last season. This was used to measure longer term performance and has been weighted by 30% for promoted teams. Therefore putting all these variables together we have:

- HT AdjGD form 5
- AT AdjGD form 5
- HT Home GS form 10
- AT Away GS form 10
- HT Home GC form 10
- AT Away GC form 10
- HT prev points
- AT prev points
- HT Market Value
- AT Market Value

Key: HT = Home Team, AT = Away Team, AdjGD = Adjusted Goal Difference, GS = Goal Scoring, GC = Goal Conceding, form 5 = rolling average from last 5 games, form 10 = rolling average from last 10 games.

Figure 2.1 represents the head of the training dataset, i.e. the first 5 games of the 2018 Premier League season. Home team, away team and season have been added for easier reading but these were not used in the model. The two target variables are home GF and away GF with all 10 predictor variables to the right hand-side.

	HT	AT	home GF	away GF	HT AdjGD form 5	AT AdjGD form 5	HT Home GS form 10	HT Home GC form 10	AT Away GS form 10	AT Away GC form 10	HT prev points	AT prev points	HT Market Value	AT Market Value
0	Man Utd	Leicester	2	1	0.410	-0.930	1.485	0.650	1.385	2.010	81	47	797,600,000	434,100,000
1	Huddersfield	Chelsea	0	3	-0.350	0.240	0.845	1.265	1.380	1.290	37	70	137,450,000	1,170,000,000
2	Newcastle	Tottenham	1	2	-0.020	0.310	1.265	0.740	1.845	0.855	44	77	244,650,000	898,600,000
3	Watford	Brighton	2	0	-0.140	-0.920	1.480	1.030	0.615	1.695	41	40	239,100,000	194,350,000
4	Arsenal	Man City	0	2	0.760	1.740	2.635	1.310	1.870	0.745	63	100	659,050,000	1,200,000,000

Figure 2.1: First 5 Rows of the Training Dataset

2.3 Variable Correlation Analysis

Figure 2.2, illustrates the relationship of the predictor variables in a correlation matrix. Only the home team variables are shown as the inclusion of the away team variables would add needless squares of zero correlation. Unsurprisingly, the most correlated variables were market value and previous points as the teams with players worth more money, in general, performed better. Perhaps also as expected was that teams with good goal scoring form often had low goal conceding form, as shown by the moderate negative correlation (-0.5). Pleasingly, our general form variable isn't strongly correlated with previous points (0.6), showing that good teams went on poor runs and vice versa. Overall, it was seen that there was substantial collinearity between the variables which can cause issues when model building as some variables may become redundant.

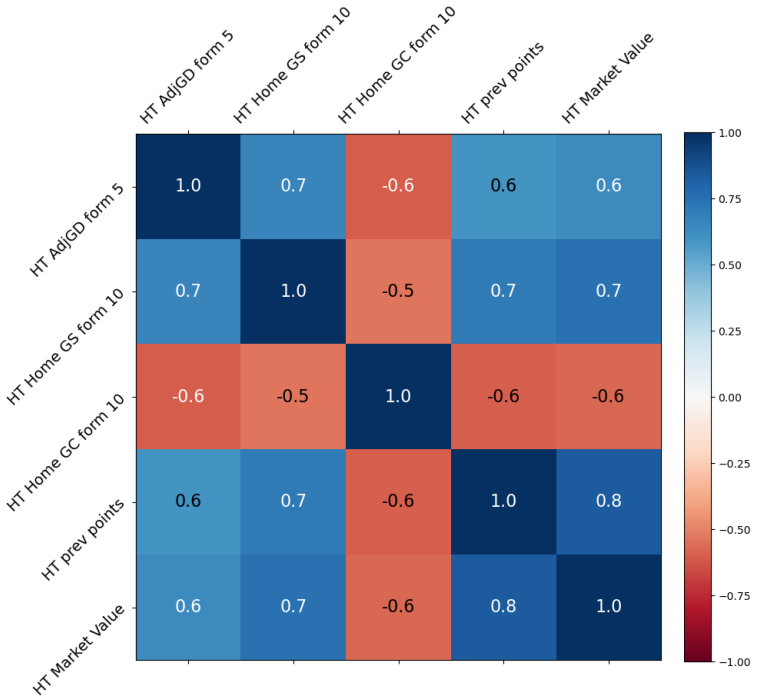


Figure 2.2: Correlation Matrix of Predictor Variables

2.4 Target Variables and Home-Field Advantage

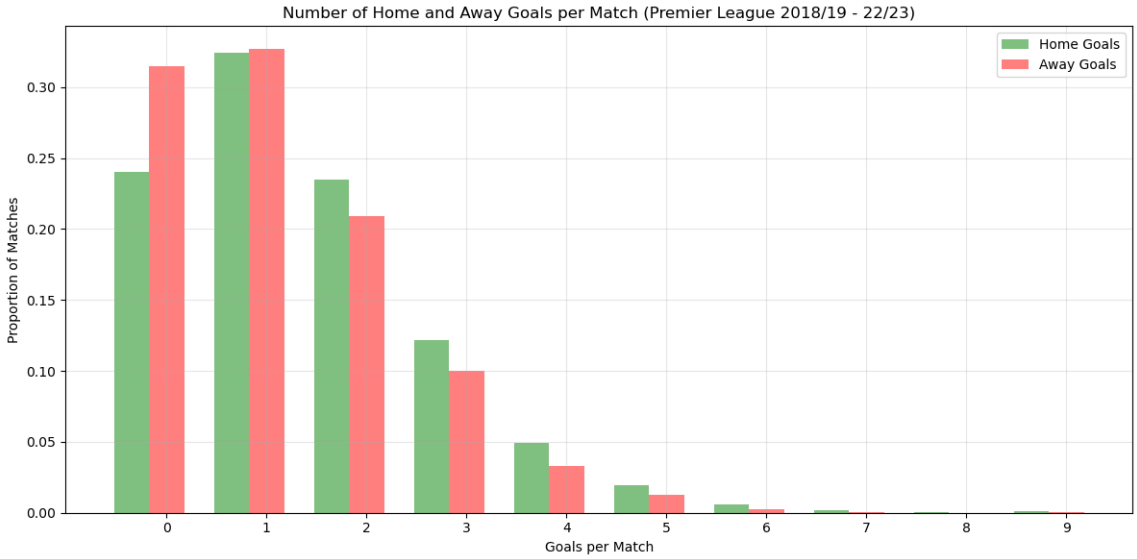


Figure 2.3: Target Variable Distributions

Figure 2.3 shows the value counts of goals scored per match for the home and away team, our target variables. In a traditional football league each team plays every other team twice, once at their stadium and once at their opponents stadium. In our data the home team scored 2+ goals in 45% of matches whereas the away team scored 2+ goals in only 38% of games. This is an example of home-field advantage which features in most sports and is used to describe the finding that home teams in sport win over 50% games played (Carron *et al.*, 2005 [5]). Many factors contribute to this finding, e.g. playing in front of a stadium full of their fans, being used to the pitch, not having to travel as far, having the home dressing room etc.

2.5 Poisson Distribution

The Poisson distribution is most often used to model count data. Usually the number of times an event occurs over a certain time interval, so it is a discrete (no half counts) and non-negative (no minus counts) distribution.

The probability mass function of the Poisson distribution includes only one parameter, λ, which is the mean number of occurrences in the given time interval, called the rate parameter. An example would be the mean number of visits to a website in one minute. The probability of k number of events occurring in a given time interval is then:

$$p_X(k) = \frac{\lambda^k}{k!}e^{-\lambda}, \quad k \in \mathbb{N}$$

where X is a Poisson random variable that takes non-negative integer values $\{0, 1, 2, \ldots\}$ [28]. For X to be considered a Poisson random variable it should also meet the following assumptions:

- Events, k, must be independent, in other words an event occurring shouldn't affect the probability of another event occurring.
- Mean = Variance, i.e. $\mathbb{E}(X) = \text{Var}(X)$
- Two events cannot occur at exactly the same time.

How does this relate to football? By pinning our target variable down to a certain distribution, i.e. Poisson, it would help in our model selection and allow us to achieve better results. Goals scored are a discrete count variable that occur over a fixed time interval (90 minutes) and multiple goals cannot be scored at the same time. However, some of the assumptions are more up for debate. Events should be independent, therefore a goal happening doesn't effect the probability of another goal happening. This could be the case in football and hard to prove either way but it is arguable that a team is more likely than normal to score if they are chasing the game late on for example. Also, as we are separating home and away goals into two distributions this means we assume they are independent of each other but this is also contentious, e.g. the home team's probability of scoring may decrease if the away team is 3-0 up.

To inspect how similar the mean and variance were, we denoted X as the number of goals scored by the home team and Y as the number of goals scored by the away team. The average number of home goals scored in our dataset, $\lambda_x = \mathbb{E}(X)$ was 1.52 and the average number of away goals, $\lambda_y = \mathbb{E}(Y)$ was 1.26. The variance of home goals, $\text{Var}(X)$ was 1.77 whilst $\text{Var}(Y)$ was 1.47.

Therefore the variance for both home and away goals was a little high but seemed close enough to continue investigating this distribution.

Figure 2.4 visualises the observed frequency of home and away goals and the expected frequency under the poisson distribution, using the average frequencies (λ_x and λ_y). The shapes were very similar, both being heavily right tailed. At each frequency of goals we observed that the same coloured dots and bars were relatively close to each other. However, the Poisson distribution did not fit well at the lower frequencies (0,1 and 2). 0 goals were scored more frequently than expected, whereas 1 and 2 goals were scored less frequently. However, overall, it looked like a good estimate.

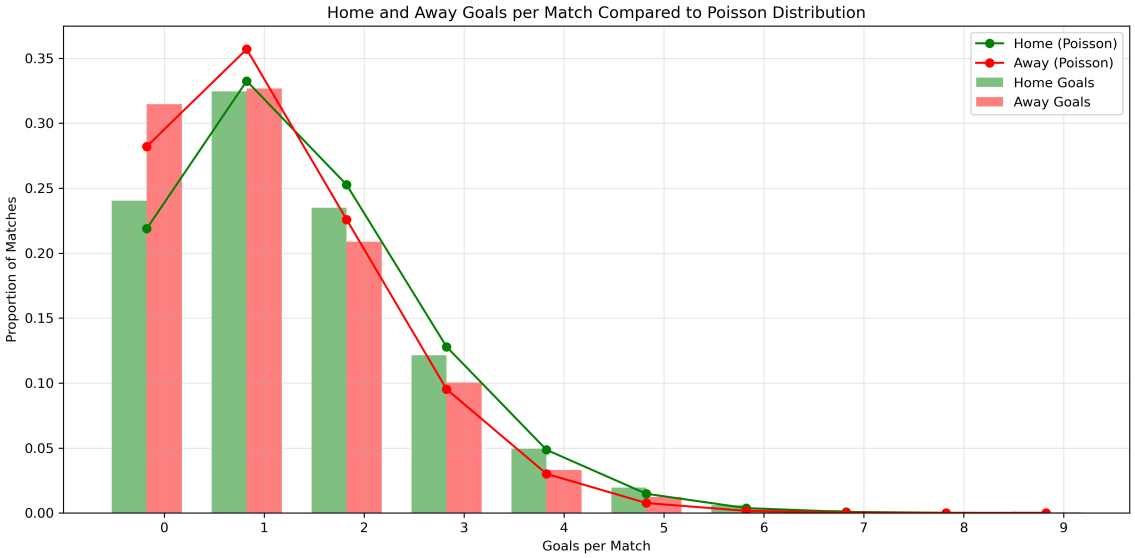


Figure 2.4: Goal Distribution Compared to Poisson Distribution

2.6 Pearson's Chi-square Goodness of Fit Test

Most literature, e.g. Azhari (2018) assumes that goals follow a Poisson distribution without mathematical proof [2], however for this paper we have gone a step further and provided some supportive statistical evidence. To test whether goal frequency can be modelled by a Poisson distribution, we used the Pearson's Chi-squared goodness of fit test. This calculates the sum of

differences between the observed and expected frequencies under the desired probability distribution. The formula is as follows:

$$\chi^2 = \sum_{i=1}^{n} \frac{(O_i - E_i)^2}{E_i}$$

Where, in our case, n was the number of goals scored, O_i the observed frequency of matches, that had each number of goals scored i, and E_i the expected frequency of matches. The degrees of freedom was then $n - p - 1$ where p was the number of estimated parameters.

Applying this to our data, using home goals, our null hypothesis H_0 was $X \sim Poisson$ and therefore our alternate hypothesis H_1 was X does not follow a Poisson Distribution. We extracted the observed frequencies from our data. Then to calculate the expected frequency we had to multiply the total number of matches (1843) by the expected proportion of matches with each number of goals according to the Poisson probability mass function, shown in Figure 2.4, using our λ_x of 1.52.

As the chi-square approximation can become poor when some of the O_i's are too small, the less frequent goal tallies $i = 4, 5, 6, 7, 8, 9$ were grouped into one bin labelled 4 or more goals. Therefore, the probabilities of these frequencies were added together to retrieve the appropriate probability for this bin. This led to 5 bins, so our degrees of freedom were $n - p - 1 = 5 - 1 - 1 = 3$ as we estimated the rate parameter of the Poisson distribution using our λ_x. The observed and expected frequencies are shown here in figure 2.5:

Goals	Observed Frequency	Poisson Probability	Expected Frequency
0	443	0.22	406
1	598	0.33	608
2	433	0.25	460
3	224	0.13	239
4 or more	145	0.07	130

Figure 2.5: Observed and Expected Home Goal Frequency

By calculating the squared sum of differences divided by the expected frequency for each amount of goals ($i = 0, 1, 2, 3, 4+$) we got the chi-squared statistic $\chi^2 = 7.7$. The critical value for the chi-squared distribution, for 3 degrees of freedom, was 7.8 at the 5% significance level, meaning that our

p-value was just greater than 0.05. Therefore, there was not enough evidence to reject the null hypothesis that the number of goals scored by the home team in our dataset followed a Poisson distribution, even though the result was not convincing. Similarly for away goals scored we got a result just under the critical value, which allowed us to continue with the Poisson distribution.

Chapter 3

Modelling Methods

3.1 Poisson Regression

3.1.1 Generalised Linear Models

Firstly we fit a Poisson Regression model to our data. Which is a type of generalised linear model (GLM), which are essentially linear models that allow the response variable to belong to any member of the exponential family of distributions instead of just the normal distribution. In this case we assume the response variable, goals scored, follows the Poisson distribution. To allow the response variable to be 'non-normal' GLMs use a link function.

3.1.2 Link Function

A link function transforms the expected value of the response variable onto an unbounded continuous scale ($-\infty$ to ∞) allowing for a linear relationship with the predictor variables. Where as before, the response variable was non-negative (0 to ∞) and could therefore not be modelled linearly. The general form of the link function is:

$$g(\lambda_i) = x_i\beta$$

Where $g(\lambda_i)$ is the link function, λ_i is the expected value of the response variable for the ith row, x_i is the vector of predictor variables for the ith row

and β the associated coefficients [12].

In Poisson regression the standard link function is the log link function as the mean response has to be positive (can't have a negative goal count). This is simply taking the natural log of λ_i. Therefore the Poisson Regression model is as shown:

$$\ln(\lambda_i) = x_i\beta$$

Where the observed values Y_i follow a Poisson distribution with parameter $\lambda = \lambda_i$ for each given vector of predictor variables x_i. For instance, each row of predictor variables will potentially have a different value of λ.

In our context we used Poisson Regression to model our rate parameter λ, the expected number of goals, as a function of our covariates. In particular, as we have just shown we actually model the log of our expected goals as a linear function of our covariates. Once we fitted the model we then predicted the mean goal count, λ_i for each game in the 2024/25 season, given the vector of predictor values x_i. For instance given some x_k values for our home and away teams, where k is the specific match, our model may return an output of 1.5 home goals, however this is our estimated rate parameter $\hat{\lambda}_k$ for the Poisson distribution of home goals in game k and not how many goals the model predicts as you cannot score half a goal. What we are more interested in is the probabilities of each home goal count that we can compute using the Poisson distribution with parameter $\hat{\lambda}_k$. For example, with rate parameter, $\hat{\lambda}_k = 1.5$, the model is predicting the home team to score 0 goals 22% of the time, 1 goal 33% of the time and so on.

3.1.3 Likelihood Function

If we continue by denoting Y as the response variables (goals), then the model takes the form:

$$\log(\mathbb{E}(Y \mid \mathbf{x})) = \boldsymbol{\theta}'\mathbf{x}$$

Where $\mathbf{x}$ denotes our vector of predictor variables and it can be seen that, as shown previously, the natural logarithm of the expected value of goals scored can be linearly modelled by the unknown parameter $\boldsymbol{\theta}$. This is a

concatenation of the intercept, (α) and coefficients, (β) which we are trying to find. Therefore, the estimated rate parameter (mean) of the predicted Poisson distribution is given by:

$$\lambda := \mathrm{E}(Y \mid \mathbf{x}) = e^{\boldsymbol{\theta}'\mathbf{x}}$$

If we denote our match dataset as m vectors of x_i (our predictor variables outlined in section 2.2) and m sets of values $y_1, ..., y_m$ then we can use maximum likelihood to find the value of $\boldsymbol{\theta}$ that maximises the likelihood of attaining the observed data (the actual number of goals scored in each match) [26]. In other words, finding the model parameters that will best fit the data. However, under Poisson Regression the traditional likelihood function has no solution and so instead we attempt to minimise the negative log-likelihood function, shown here:

$$-\log L(\boldsymbol{\theta} \mid X, Y) = \sum_{i=1}^{m} \left(y_i \boldsymbol{\theta}' \mathbf{x}_i - e^{\boldsymbol{\theta}'\mathbf{x}_i} \right)$$

This is then our convex loss function we can then minimize to find the optimal value of $\boldsymbol{\theta}$ [31].

3.1.4 Independence

In order to use Poisson Regression in the context of a football match we had to fit two separate models, one for home goals (X) and one for away goals (Y). This is because if we were to just model total goals scored, we wouldn't know who won as we don't know how many each team scored. Another way to approach this problem as used by Karlis and Ntzoufras (2009) could be to model the difference between the number of goals scored by each team [10]. For instance the calculation would be $X - Y$ and therefore we could work out the result depending on whether the output was positive, negative or zero. This is called the Skellam distribution, i.e. the difference between two Poisson distributions. However, this wouldn't allow us to work out how many each team scored. For example, if the output was 2, we know the home team has won by two goals but we don't know the score (whether it was 2-0, 3-1 etc...). This would then mean we can't use variables like goals scored

when simulating the season which should be an important part of predicting the score of a football match.

However, using two models came at a cost. This didn't allow us to account for any dependency between home goals and away goals. Which on the surface may sound like a poor assumption as they are mutually exclusive at the event level, i.e. if A denotes the event of a home goal and B the event of an away goal then $P(A \cap B) = \emptyset$, they cannot both happen at the same time. However, if we instead return to treating them as random variables, with X denoting the random variable home goals and Y the random variable away goals over a 90 minute football match, then they are said to be independent if the combined random variable (X, Y) has a joint cumulative distribution function:

$$F_{X,Y}(x, y) = F_X(x)F_Y(y) \text{ for all } x, y$$

Where $F_X(x)$ and $F_Y(y)$ are the cumulative distribution functions (CDFs) for X and Y (home and away goals) [27], shown in figure 3.1. We know that the two random variables are not strictly independent but how much dependency is there, i.e. how much is the difference between the product of their CDFs and their joint CDF? Using the joint CDF, shown in figure 3.2 let's take the example that both teams score 2 goals, therefore $x = 2, y = 2$. Here, $F_X(2) = 0.7998$ and $F_Y(2) = 0.8502$ and their product is therefore 0.68. If we compare this to the actual joint CDF $F_{X,Y}(2, 2) = 0.67$, we see that there is a slight difference of 0.01. Trying a different example with 0 home goals and 3 away goals, we get an even smaller difference of 0.005. Therefore this shows a very small dependency but not one big enough to ignore using this model.

3.1.5 Hyper Parameters

Using Scikit Learn's Poisson Regressor class a Poisson regression model was fitted to both home and away goals [20]. However, we had some model parameters (different from θ mentioned before) to choose first, called hyper parameters. The first parameter was whether or not to fit an intercept to the model, denoted α in section 3.1.3. Essentially, it decides whether or not to add any bias to the model, acting as a starting point. This is because it is the mean value of the response variable when all independent variables

Goals (x,y)	Fx(x)	Fy(y)
0.0	0.2404	0.3147
1.0	0.5648	0.6413
2.0	0.7998	0.8502
3.0	0.9213	0.9506
4.0	0.9707	0.9837
5.0	0.9902	0.9962
6.0	0.9962	0.9989
7.0	0.9984	0.9995
8.0	0.9989	0.9995
9.0	1.0	1.0

Figure 3.1: Cumulative Distribution Functions, $F_X(x)$ and $F_Y(y)$

Home Goals (X)

Away Goals (Y)	0	1	2	3	4	5	6	7	8	9
0	0.0619	0.1546	0.236	0.2773	0.299	0.3093	0.3114	0.3131	0.3136	0.3147
1	0.1346	0.3332	0.4954	0.5838	0.6202	0.6343	0.6381	0.6397	0.6403	0.6413
2	0.1926	0.4645	0.6701	0.7802	0.8237	0.8416	0.8464	0.8486	0.8492	0.8502
3	0.2225	0.5263	0.752	0.8725	0.9213	0.9409	0.9468	0.949	0.9495	0.9506
4	0.2339	0.5518	0.7835	0.905	0.9544	0.974	0.9799	0.9821	0.9826	0.9837
5	0.2387	0.5616	0.796	0.9175	0.9669	0.9864	0.9924	0.9946	0.9951	0.9962
6	0.2393	0.5638	0.7987	0.9202	0.9696	0.9891	0.9951	0.9973	0.9978	0.9989
7	0.2398	0.5643	0.7992	0.9208	0.9702	0.9897	0.9957	0.9978	0.9984	0.9995
8	0.2398	0.5643	0.7992	0.9208	0.9702	0.9897	0.9957	0.9978	0.9984	0.9995
9	0.2404	0.5648	0.7998	0.9213	0.9707	0.9902	0.9962	0.9984	0.9989	1.0

Figure 3.2: Joint Cumulative Distribution Function, $F_{X,Y}(x, y)$

are set to 0. Confusingly, we also had the alpha hyper parameter which acts as a regularisation term to mitigate over fitting due to collinearity between independent variables. Formally this changes the loss function slightly to instead maximising:

$$\sum_{i=1}^{m} \left(y_i \boldsymbol{\theta}' \mathbf{x}_i - e^{\boldsymbol{\theta}' \mathbf{x}_i} \right) - \lambda \|\theta\|_2^2$$

Here λ is the hyper parameter we can change to control the strength of the regularisation and $\|\theta\|_2^2$ is the Euclidean norm of our parameter vector θ. Overall this term essentially penalises large coefficients to prevent over fitting. Over fitting occurs when a model fits to the training dataset too well and will, as a consequence, perform badly on the test dataset. Large or complex coefficients can often lead to this problem so the goal of the regularisation is to find the value of λ that keeps coefficients small but also fits the data well. The last hyper parameter was the number of iterations the solver algorithm used to optimise our problem. Large numbers of iterations can be useful in finding the most optimal parameters but it can also slow down computation and cause over fitting.

3.1.6 Tuning and Grid Search

Instead of just randomly choosing our hyper parameters we used a grid search algorithm to optimise them by picking the values that minimised our loss function. It did this by exhaustively iterating over each possible set of parameters and returning which set achieved the smallest value. The advantages of using a grid search algorithm is that it works with all cost functions and is very easy to implement, as shown in Appendix A.11. However, there is no guarantee that you actually find the set of parameters which minimises the loss function, i.e. you might only find a local minima instead of a global one. This is because the algorithm only iterates over the values you've given and therefore there could be some more optimal values you haven't chosen. Also, for a large set of parameters it can become very slow due to its exhaustive nature. However, for our model we have a small amount of parameters so it was appropriate to use.

A range of values were picked from 0 to 1 in 0.1 increments for the regularisation term alpha, 100, 200 and 500 iterations and either true or false for

the intercept. Before running the grid search algorithm you have to choose a scoring system to evaluate the performance of each cross-validated model. Here, the mean squared error (MSE) was used which measured the difference between the model and our actual data (the lower score the better). In context this took the difference between expected mean number of goals and actual goals scored. Using Scikit Learn's GridSearchCV class [18], the `grid_search` function split up the training data into 5 folds (the default grid search setting) and each fold was used as a test whilst the other 4 are used to train the model. This therefore meant each set of parameters was tested 5 times, with the average score returned.

For both models the optimal choice of parameters was an alpha of 0.2, 100 iterations and to fit an intercept. The home goal model had a mean cross-validated MSE of 1.48 and 1.32 for the away goal model. Interestingly, the intercept values we got, α, for the home and away models were 0.36 and 0.19 respectively. This showed that as a starting point the model was already expecting on average the home team to score more goals. A regularisation factor of 0.2 represented moderate regularisation which slightly penalised large coefficients.

3.1.7 Monte Carlo Simulations

We then used the Poisson Regression models to make predictions on the test data and see how it performed. The test dataset was the most recently played Premier League season (23/24). Therefore, it had 380 rows of matches to predict the home and away goals for, with each team's form based data at that point in the season. This changed when we forecast the matches for next season as we didn't know this data, i.e. the matches hadn't happened yet. For each row the models predict the mean number of goals scored, however, as mentioned in section 3.1.2, this is just the mean response and not the model's actual prediction for goals scored. Instead used this mean response as the rate parameter for individual Poisson distributions which gave useful predictions for the likelihood of each team scoring an amount of goals.

As an example, the prediction for the opening game of the season is shown in Figure 3.3. The model predicted the average number of goals for Burnley as 0.79 and 2.45 for Manchester City. This gave the resulting Poisson distributions for the expected probability of each team scoring each number of goals. Using these probabilities to generate a score matrix like in Figure 3.4,

a random scoreline was then generated. The most likely scoreline is 2-0 to Manchester City at 11.7%. Once all 380 games had been randomly generated we then worked out the models prediction for the final league table. However, to obtain a better estimate of its true predictions and to eliminate the random, low scoring, aspect of football, the Monte Carlo method of repeated random sampling was used to obtain 10,000 simulated seasons, which led me towards the true prediction. This process was wrapped in the function `simulate_all_games`, shown in Appendix A.8 which returned the average simulated league table.

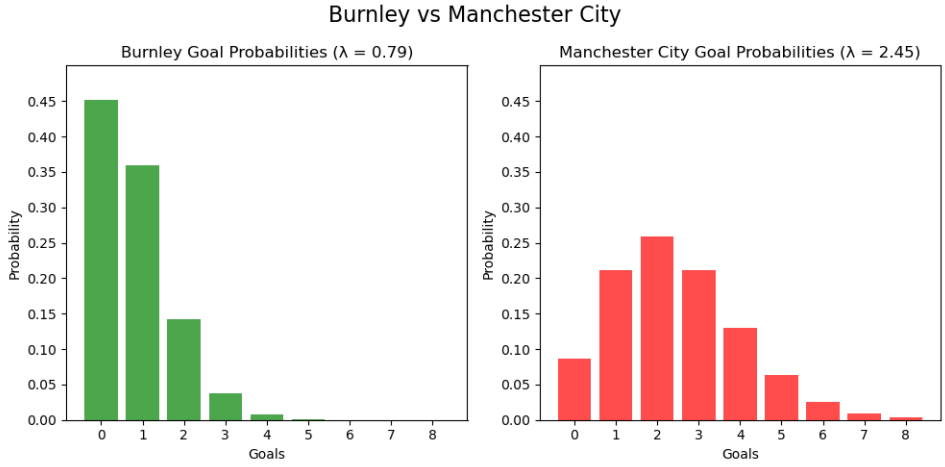


Figure 3.3: Burnley vs Manchester City Goal Distributions

Lastly before we get on to the results, there was one big change in the Premier League last season. This was the amount of time being added on at the end of games had increased. Therefore as each match on average was going on for longer it resulted in more goals being scored, which we needed to adjust for in the models. Using the data found on Statista (2023) we created the table shown in figure 3.5 [23]. We averaged the time per game over the training data, which was 97 minutes and 16 seconds and compared it to the test data (101 minutes and 30 seconds) and it was a 4.35% increase. Therefore, as a simple approach the mean predictions from the model where multiplied by 1.0435 to account for this.

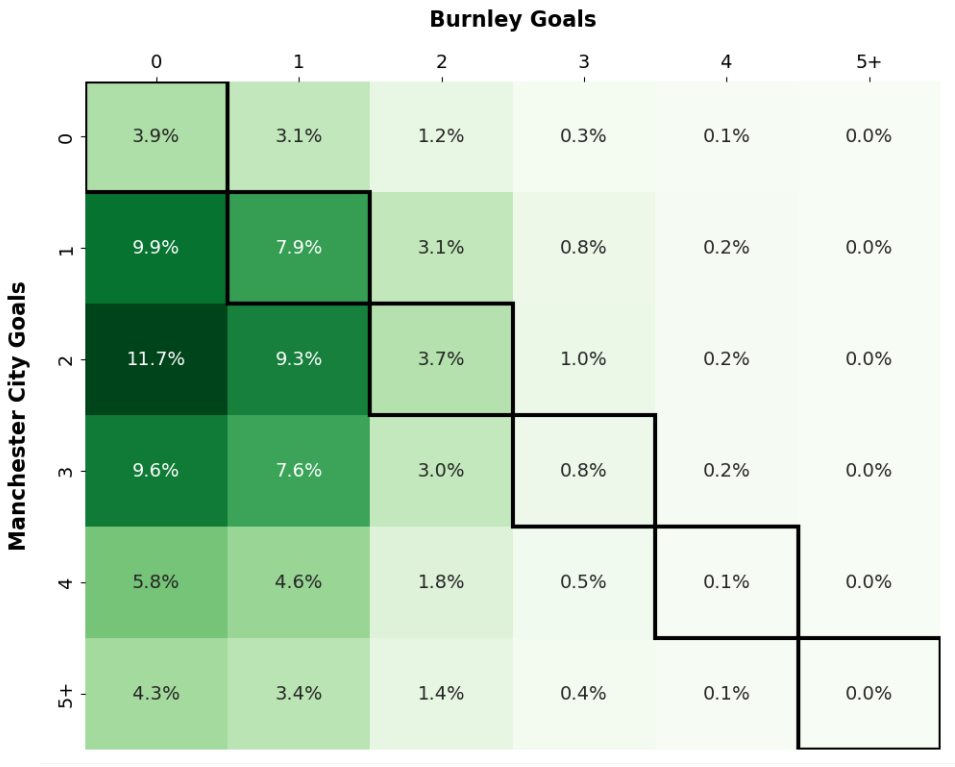


Figure 3.4: Burnley vs Manchester City Score Probabilities

Season	Average Match Time
2018/19	96.4
2019/20	97.32
2020/21	96.34
2021/22	97.29
2022/23	98.06
2023/24	101.3

Figure 3.5: Average Match Time per Season in Minutes

3.1.8 Results

To present the simulations like the actual final Premier League table, each team's average amount of points was calculated and sorted in descending order. The table is shown in Figure 3.6. Note that Everton and Nottingham Forest both had point deductions in the 23/24 season so these have been accounted for.

The left side of the table shows our predictions and the right side shows how the Premier League actually finished in the 23/24 season. We would expect the model to be pretty accurate as we have the actual form based data for each team throughout the season. Immediately, it predicted the correct winner (Man City) and also the 2nd and 3rd place teams. The points are sightly lower than their actual tallies but this is to be expected as the teams that finish high up the table are likely to outperform their data. The first team we got wrong was Chelsea in 4th when they actually finished 6th but we only over predicted by 2 points on average. Point predictions for Man United and Newcastle United were bang on, whilst the model was slightly off in Tottenham and Aston Villa's case, who both outperformed their expected data. The middle of the table is so tight points wise that it is hard to be totally accurate but the points seemed close enough. The only teams the model got significantly incorrect were Brentford (10 point over prediction), who on xGD were actually the 8th best team in the league, suggesting they were unlucky to finish as poorly as they did. West Ham were under predicted by 7 points, however, they actually over performed their expected data (16th lowest xGD). The bottom three relegated teams were all predicted but not in the exact order. Overall the core of the prediction looks good, with both ends of the table being correct without any signs of over fitting the test data, i.e. predicting extremes like Sheffield Utd to get as little as 16 points.

3.2 Logistic Regression

3.2.1 Binary Logistic Regression

The second model was a type of logistic regression, however, to assist in the eventual, quite complicated model definitions we will start by explaining the most simple type of logistic regression. Binary logistic regression is a special type of regression which actually performs a classification task. It is another

Poisson Model Predicted vs Actual 23/24 Premier League Table

	Predicted			Actual		
Pos	Team	Points	Points	Team	Pos
1	Manchester City	87.7	91	Manchester City	1
2	Arsenal	82.3	89	Arsenal	2
3	Liverpool	74.5	82	Liverpool	3
4	Chelsea	65.0	68	Aston Villa	4
5	Aston Villa	60.6	66	Tottenham	5
6	Newcastle Utd	60.5	63	Chelsea	6
7	Tottenham	59.9	60	Newcastle Utd	7
8	Manchester Utd	59.8	60	Manchester Utd	8
9	Brighton	51.3	52	West Ham	9
10	Brentford	49.4	49	Crystal Palace	10
11	Crystal Palace	46.7	48	Brighton	11
12	West Ham	45.2	48	Bournemouth	12
13	Fulham	44.7	47	Fulham	13
14	Bournemouth	44.0	46	Wolves	14
15	Wolves	42.9	42	Everton	15
16	Nott'ham Forest	40.6	39	Brentford	16
17	Everton	38.7	32	Nott'ham Forest	17
18	Burnley	35.3	26	Luton Town	18
19	Luton Town	31.1	24	Burnley	19
20	Sheffield Utd	28.4	16	Sheffield Utd	20

Figure 3.6: Poisson Model Predicted 23/24 Premier League Table

form of GLM where the response variable is assumed to be the outcome of a Bernoulli trial. This is a unique case of the Binomial distribution where there is only one trial, i.e., the response variable either takes value 1 with probability p or 0 with probability $1 - p$. The response variable here is a probability, p, which takes values between $(0, 1)$ and therefore cannot be modelled linearly, similar to the Poisson response variable. As explained in section 3.1.1, GLM's need a link function to convert the response variable onto a continuous scale, i.e. $(-\infty, \infty)$ to allow for a linear relationship with the predictor variables. Using our probability notation, for this type of model the chosen link function is the logit link function, which has the following formula:

$$\operatorname{logit} p = \ln\left(\frac{p}{1-p}\right)$$

The logit function works by first calculating the odds of the response variable taking value 1 given the predictor variables. Odds have a unique relationship with probability but are different things. Odds of the response variable taking value 1 is easiest explained as a ratio. The ratio of the probability the response takes value 1 compared to the probability of it taking 0, this is the therefore the $p \div 1 - p$ we see above. As p and $1 - p$ will always be positive, so will the odds, therefore the logit function then takes the logarithm of the odds to allow the response to be negative, matching the range of values the right side of the equation could take. This whole process is also called taking the 'log-odds'. To emphasise and use previous notation this means the logit of the expected value of the response variable Y_i given predictor variables x_i can be modelled linearly in the following way:

$$\operatorname{logit}\left(\mathbb{E}[Y_i \mid \mathbf{x}_i]\right) = \operatorname{logit} p(x) = \ln\left(\frac{p(x)}{1-p(x)}\right) = \boldsymbol{\beta} \cdot \mathbf{x}_i$$

Where the probability, $p(x)$ and the regression coefficients, $\boldsymbol{\beta}$ are both unknown.

To interpret the log-odds and make meaningful predictions we used the inverse logit function, called the standard logistic, which converts log-odds back into a probability. The standard logistic function formula is:

$$f(x) = \frac{1}{1 + e^{-x}}$$

Where x in this context was be the log-odds of the response variable, which as previously shown is equal to the dot product of our regression coefficients and predictor variables, $\boldsymbol{\beta} \cdot \mathbf{x}_i$. Therefore, plugging this in, we can then find the expected value of the response variable using the inverse logit function:

$$\mathbb{E}[Y_i \mid \mathbf{x}_i] = p(x) = \text{logit}^{-1}(\boldsymbol{\beta} \cdot \mathbf{x}_i) = \frac{1}{1 + e^{-\boldsymbol{\beta} \cdot \mathbf{x}_i}}$$

One of the advantages of using the logistic function, is that it is really easy to go between log-odds and probabilities. In this case we were interested in probabilities. The function is also sometimes called a sigmoid function (S-shaped curve) as when it is graphed we get a nice S-shaped curve shown in 3.7 [30]. It can now be seen how the function converts the x continuous x axis to probabilities on the y axis.

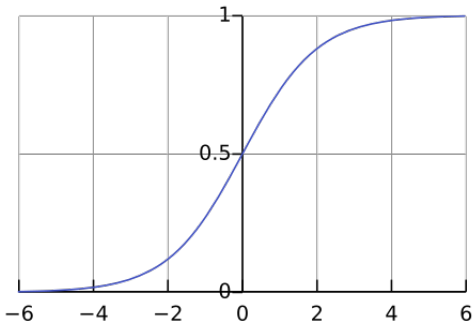


Figure 3.7: Standard Logistic Function

3.2.2 Multinomial Logistic Regression

However goals are not a binary response variable as we cannot split goals into two categories. Instead we allocated each number of goals as a category up to 9 goals (categories). To accommodate for the response variable no longer being a member of the Bernoulli distribution we instead needed to use multinomial logistic regression, assuming the response variable is categorical.

In binary logistic regression we only had two categories and two probabilities where as here we have K number of categories (9) and need K number of probabilities, one for each category. When $K > 2$, i.e. no longer binary, the regression becomes more complicated. We have to adapt the linear predictor function as we now have multiple sets of regression coefficients, $\boldsymbol{\beta_n}$, for $n = 0, 1, ..., K$, where $\boldsymbol{\beta_n}$ is the regression coefficient assigned to category k. Therefore this leaves us with a new linear predictor function:

$$\ln\left(\frac{p_n(x)}{p_0(x)}\right) = \boldsymbol{\beta_n} \cdot \mathbf{x_i}$$

The left-hand side of this equation is the log-odds of the response variable being in category n. Where $p_n(x)$ is probability that the response variable is assigned category n given the predictor variables $\mathbf{x_i}$ and $p_0(x)$ is the sum of the probabilities that the response variable is assigned one of the other categories. Notice that this is the same as before but it was denoted $1 - p$).

To convert the log-odds of our response variable back to probabilities we cannot use the logistic function as we no longer have a single log-odds response. Instead we used generalisation of the logistic function called the softmax function which handles multiple inputs. This is slightly different to the logistic function as it takes the exponential of the log-odds and divides it by the sum of the log-odds for all categories. Therefore the probability of the response variable Y_i belonging to category n derived from the log-odds x_n is shown here:

$$P(Y_i = n) = \text{softmax}(n, x_1, \ldots, x_K) = \frac{e^{x_n}}{\sum_{i=1}^{K} e^{x_i}}$$

Where the softmax function takes a vector of log-odds components, $(x_1, \ldots, x_K)$, and transforms each one into a probability in the interval $(0, 1)$, where the sum of probabilities equals 1. The probability of the response variable being in category n will then be the nth element of the output vector [29]. For example say we had limit goals (K) to 1, 2 or 3 and we had the vector of log-odds $(x_1, x_2, x_3) = (1.5, 0.3, -3.7)$, it would convert this to the probabilities $(0.765, 0.231, 0.004)$. Therefore from this we can interpret that the model thinks the team has a 76.5% probability of scoring 1 goal.

3.2.3 Likelihood Function

The regression coefficients are unknown, along with the probabilities (response variable), therefore like in Poisson regression we need to estimate them using MLE. Again this is an iterative process that tries different sets of coefficients and chooses the set that maximises the likelihood of the model producing the observed data (the actual number of goals scored). The likelihood function for this problem is defined as:

$$L = \prod_{i=1}^{s} P(Y_i = y_i) = \prod_{i=1}^{s} \left(\prod_{n=1}^{K} P(Y_i = n)^{\delta_{n,y_i}} \right),$$

where the observed response values y_i are equal to one of the categories $0, 1, ..., K$ for each row of observations $i = 1, ..., s$ and the index n denotes each category. δ_{n,y_i} is called the Kronecker delta, a type of indicator function which essentially maps a value to 1 or 0 depending on some common notation. In this case it gives a value of 1 if our observed value, y_i equals the category n and 0 if it equals any other category, as shown here:

$$\delta_{n,y_i} = \begin{cases} 1 & \text{if } n = y_i \\ 0 & \text{otherwise} \end{cases}$$

Like with Poisson regression, instead of maximising the likelihood function above we instead minimise the negative log-likelihood function shown [25]

$$-\log L = -\sum_{i=1}^{s} \sum_{n=1}^{K} \delta_{n,y_i} \log(P(Y_i = n)).$$

3.2.4 Multi output Classification

Assuming home and away goals are categorical variables that can take values from 0 to 9, we are then dealing with multiple classes and multiple output variables. In order to predict both of these with logistic regression we had to use multi output classification. This extends standard classification tasks to predict multiple response variables through fitting one classifier per target variable. This allows the model to predict a series of responses whilst still

being trained on the same predictor data matrix. Unfortunately, this still isn't capturing any dependency between the target variables (home and away goals) but as we have shown this is a very minimal amount.

3.2.5 Hyper Parameter Tuning

As Scikit Learn's multi class logistic regression doesn't natively support multi target classification we used their MultiOutputClassifier class to wrap round our model. This, as mentioned above, fit one logistic regression classifier to each target variable [21]. Logistic regression had some hyper paramters to choose. Again similarly to Poisson the main ones we focussed on were whether or not to fit an intercept, the max number of iterations and the strength of the regularisation parameter, denoted C here instead of alpha [22]. These concepts have been explained in section 3.1.5 and the only change is that instead of alpha we had C which was the inverse of regularisation strength. Therefore, the smaller the value of C, the stronger the penalty is on the model having large coefficient values.

For the Poisson regression model we did a simple grid search algorithm using the negated mean square error as our scoring mechanism to find our hyper parameters. In this case the response variable was the expected mean output where as for logistic regression the response variable was the category with the largest probability. To overcome this, Appendix A.12 shows a manual grid search algorithm which instead takes the probabilities of each goal category and finds the mean value, akin to what we have with the Poisson model. This meant we could use the MSE to good effect and make a meaningful judgement to which hyper parameter values were the best. The chosen model, with the lowest MSE on average, had the values: C = 0.1, iterations = 200 and to fit an intercept.

3.2.6 Results

Again we used the Monte Carlo method of simulating the test dataset (23/24 season) 10,000 times. We used the probabilities of each number of goals according to the logistic regression predict proba method which outputted the probability estimates for all classes. However, to account for the 4.35% longer playing time the cumulative probability distribution was slightly shifted to increase the likelihood of higher goal counts. Taking each teams average

points and accounting for point deductions, the simulated table is shown in Figure 3.8:

As before the right side shows the actual 23/24 Premier League table and the left side now shows the predictions of the logistic regression model. Similarly, we predicted the top 3 teams correct. However, this time the points were closer to their actual tallies. Interestingly, Chelsea were predicted on average to get far more points and comfortably sit in 4th. This suggestted an over-reliance on the market value variable when predicting each game as Chelsea have the 3rd biggest market value. They also finished on 44 points last season so to predict 70 points this season suggests that the previous points variable may be somewhat ineffective. Point predictions for the rest of the table were very similar to the Poisson model with Brentford and West Ham being the only teams significantly off. The model again correctly predicted the bottom three relegated teams but like, at the top end, closer to their actual points. On face value this looked like it could be better than the Poisson model but the accuracy of predictions at either end of the table may suggest some over fitting.

Logistic Model Predicted vs Actual 23/24 Premier League Table

	Predicted			Actual		
Pos	**Team**	Points	Points	**Team**	**Pos**
1	**Manchester City**	92.3	91	**Manchester City**	**1**
2	**Arsenal**	86.0	89	**Arsenal**	**2**
3	**Liverpool**	77.3	82	**Liverpool**	**3**
4	**Chelsea**	70.0	68	**Aston Villa**	**4**
5	**Tottenham**	62.1	66	**Tottenham**	**5**
6	**Aston Villa**	61.5	63	**Chelsea**	**6**
7	**Manchester Utd**	60.6	60	**Newcastle Utd**	**7**
8	**Newcastle Utd**	59.7	60	**Manchester Utd**	**8**
9	**Brighton**	50.1	52	**West Ham**	**9**
10	**Brentford**	47.8	49	**Crystal Palace**	**10**
11	**Crystal Palace**	45.8	48	**Brighton**	**11**
12	**West Ham**	45.0	48	**Bournemouth**	**12**
13	**Bournemouth**	42.9	47	**Fulham**	**13**
14	**Fulham**	42.8	46	**Wolves**	**14**
15	**Wolves**	41.9	42	**Everton**	**15**
16	**Nott'ham Forest**	40.7	39	**Brentford**	**16**
17	**Everton**	37.4	32	**Nott'ham Forest**	**17**
18	**Burnley**	33.8	26	**Luton Town**	**18**
19	**Luton Town**	28.8	24	**Burnley**	**19**
20	**Sheffield Utd**	26.6	16	**Sheffield Utd**	**20**

Figure 3.8: Logistic Model Predicted 23/24 Premier League Table

Chapter 4

Model Selection

4.1 Model Comparison

4.1.1 Model Discussion

The Poisson regression model allowed us to build on previous literature, for example, Maozad *et al.* (2022) created a simple but successful model built on attack and defence ratings with a home advantage intercept [11]. However, we built on this approach by adding more complex covariates and avoided assuming the home team had an advantage. Instead we used the form of each team depending on the venue, e.g. the away team may actually perform better away from home. We did have to assume goals scored followed a Poisson distribution when using this model and were also independent which we proved to be unconvincing assumptions. Furthermore, a key assumption to the Poisson distribution was that mean and variance are equal, however, we calculated in section 2.5 that variance was greater than the mean for home and away goals. This can cause over dispersion in the model, with the variance of the response variable being greater than assumed.

The advantage of instead using the logistic model is we did not assume goals were Poisson distributed but instead treated the number of goals as a categorical variable. This is something different to most literature and allowed us to make less assumptions about the predictor variables, i.e. no linear relationship. However, it is best used when collinearity is low between predictor variables which is shown in section 2.3 to not be totally accurate. Addition-

ally, the categories should have no rank which may not be the case as there is some increasing order in goals scored. Therefore, an ordinal logistic model may be more suitable as it assumes some order within the categories.

4.1.2 Repeated K-Fold Cross Validation

To assess which model is the best we calculated how far off the actual amount of home and away goals each model is on average. We achieved this by taking the mean squared error of the difference between actual and predicted home goals and away goals. Then the lower average of those two MSE values pointed us towards which model to use. However, to prevent picking a model that overfits to the test dataset, the data was shuffled to test the model on different parts of the dataset [19]. This is called repeated k-fold cross validation, shown in Appendix A.13 with the function `kfold_cv`.

To prevent data leakage, i.e. the test data being used in the training process, we used the training dataset. The function worked by splitting the training dataset into k folds (parts), where each fold was used as a dummy test dataset and the other k - 1 folds were used to train the model. Then the process is repeated n times to mix up the folds of data that we are testing the models on. 5 splits (k) were used as we have 5 season of data in the training dataset so each fold will act as a dummy season, even though each match will be randomly selected across the 5 years of data. We repeated this 10 times (n), which therefore tested our models on 50 different sets of data. The mean MSE score across these 50 sets were then compared.

The score for the Poisson Model was 1.402, meaning on average the models squared prediction of home and away goals was 1.402 off the actual home and away goals. The score for the logistic model was very close but slightly higher at 1.405, suggesting there was not much predictive quality between the two models but the Poisson model was the one to choose, which we used for our forecast.

4.2 Feature Analysis - Bootstrap

To evaluate which predictor variables the Poisson model deemed most effective in predicting goals we used a bootstrapping procedure. This is similar to cross-validation, however, we did need a test dataset as we are not assessing

the model's performance but rather which coefficient values the model used when fitting the data. These values are the most optimal values of observing the given data. Bootstrapping involves creating 'bootstrap' datasets which are made from re sampling the original dataset with replacement, i.e. the same row (match) can be chosen more than once. We then fit the model on each 'new' bootstrap dataset and recorded the weight coefficient values, this process is shown in Appendix A.10. The box plots in Figure 4.1 represent the weight values for each predictor variable. This shows the average values and the range they have taken in the bootstrap datasets.

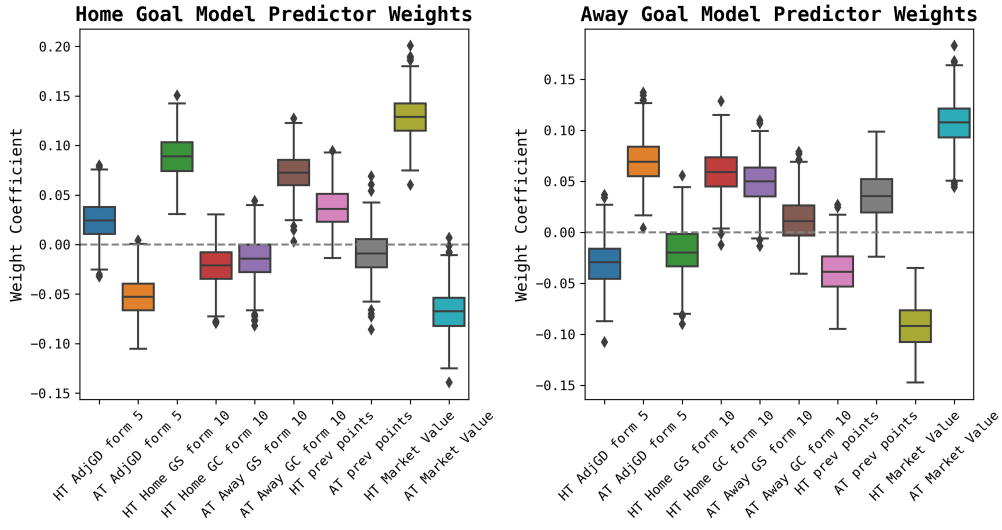


Figure 4.1: Weight Coefficients for Home and Away Goals

Immediately, we see some symmetry between the models due to the variables having inverse effects on home and away goals. For example, the home teams form positively effects home goals but negatively effects away goals. By looking at the right side of both plots we see that market value has the largest absolute coefficient and, therefore, seems to have the biggest effect in predicting goals. This is perhaps unsurprising as this is the variable assessing the teams overall skill. However, an over-reliance on this may occur which could cause poor predictive ability, i.e. just predicting the team with the highest market value to win every game which would be a pretty useless model. Some variables have less of an effect, i.e. the home goal model doesn't deem Away GS Form or Home GC Form very effective, with coefficients near

0. This is the same for the away goal model with Away GC Form and Home GS Form. These findings are unsurprising as, for example, the goal scoring form of the away team shouldn't have an effect on the number of goals the home team scores.

To reduce the reliance the model has on market value, we explored how the model would perform with a higher penalty term, i.e. less accepting of high coefficient values. Additionally, we removed the mentioned variables above that seem to have little effect on their respective models. To test the performance, we, again, used the k-fold validation procedure outlined in section 4.1.2. After trying a range of alpha values, 0.65 seemed to be the sweet spot, with the cross-validation score only slightly increasing to 1.403. Figure 5.1 shows how the changes to the model have impacted the coefficient values of the predictor variables on the bootstrapped datasets. Variables are now more compact in their results, and therefore all have a moderate effect on the predictions. The market value coefficients have significantly reduced on average which reduces the reliance. Previous points is now more effective compared to before, which may have been due to how it correlates with market value. This is because when two predictor variables are highly correlated, a model is likely to treat them as the same and rely on just one of the variables are giving us the same information.

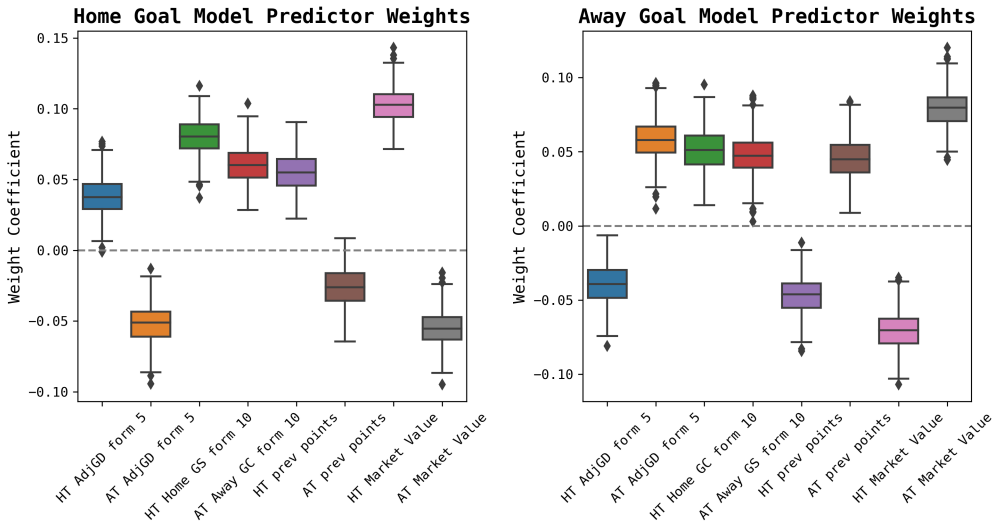


Figure 4.2: Adjusted Weight Coefficients for Home and Away Goals

Chapter 5

Forecast

The first and second games of the 24/25 Premier League season have been played but these games will not be used in my predictions, essentially we will be going into the season blind, knowing none of the results so far. The only predictor variables we have are the for each team going into the first game, from the previous season.

5.1 Approach

To solve the problem we used a very similar approach to how we tested the model on the 23/24 season. The season was simulated 10,000 times, however, this time we do not know the actual result of the game. For example before we would predict the score of a game but the form of each team for the next game was calculated on what the actual score was and not the predicted result. Therefore, when forecasting out of sample data, where we did not have the target variable, the simulations were instead run 'hot', i.e. each simulated result will impact the form variables for the next game, this process is shown in Appendix A.9. This is furthering other papers that have used these types of simulations, i.e. Saraiva's (2016) research that consisted of only 1,000 simulations and weren't run 'hot' [17]. This type of approach caused a major range of predictions in the simulations as a team can pick up an unlikely win and then their chances of winning the next game increased, causing a run of good results, finishing in a higher position than expected. Or the opposite can happen where a team goes on a bad run of form.

5.2 Results

The results table consists of the mean prediction for each team, i.e. mean number of goals scored/conceded and points over the 10,000 simulations. As far as the actual predictions go, shown in Figure 5.1, it looked standard with the three promoted teams all going back down and the top three teams in the same order as last season. These were probably good predictions, for example massive companies like Opta and betting companies also make these predictions [14]. Interesting predictions were that Chelsea convincingly get into 4th position by almost 10 points on average. This was something I assumed may happen when using market value as one of the predictor variables due to their massive squad. This wasn't a particularly unlikely prediction as Chelsea should compete for top 4 with the amount of money they have spent. However the context behind their squad is that it is made up of over 40 players when a team can only include 25 in the actual squad, so a lot of their squads market value will rarely be seen on the pitch and therefore may skew their predicted position. This is something a more 'advanced' model may be able to pick up on and could be something to look into in the future.

Other interesting predictions were that Wolves and Fulham were the two teams predicted to be most involved with the promoted teams in the relegation scrap. The model has likely predicted this due to both teams losing key players and finishing poorly last season. There was a clear gap between the top 8 teams and the rest of the league, but Crystal Palace came at the top of the other teams (9th). Perhaps due to their incredible form at the back end of the last season. If they hadn't lost one of their key players the model may have predicted them to do even better. Manchester City were predicted to win the Premier League for the fifth time in the row, however, like last season Arsenal were predicted to run them close, finishing just 3 points behind on average.

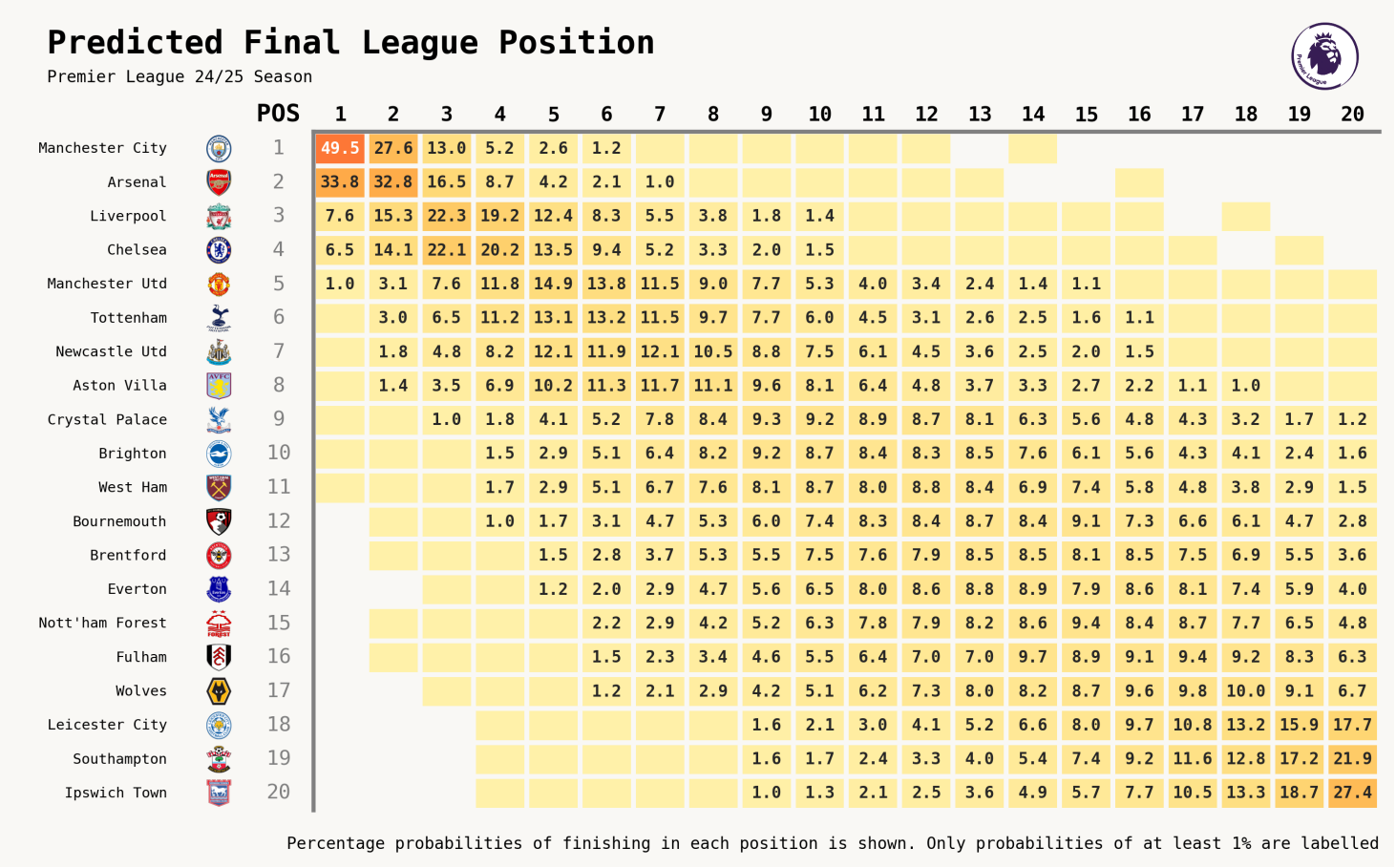
In order to give more explanation of how the 10,000 simulations work, figure 5.2 visualises how likely each team was to finish in each position (1-20). This was heavily inspired by Opta's (2024) work, in plotting a heatmap relating to the probabilities of each teams final position [14]. Any highlighted squares mean that the team had finished in that position in at least one of the simulations, i.e. West Ham came in first position, winning the league, in at least one simulation. Only probabilities of over 1 have been labelled,

Average Predicted 24/25 Premier League Table

Pos	Team	GF	GA	GD	Points
1	Manchester City	103.8	39.6	64.2	85.6
2	Arsenal	94.8	41.0	53.8	82.0
3	Liverpool	81.0	47.4	33.6	72.2
4	Chelsea	79.4	47.7	31.8	71.3
5	Manchester Utd	67.4	54.0	13.4	61.4
6	Tottenham	67.1	54.4	12.8	61.0
7	Newcastle Utd	64.5	56.2	8.2	58.3
8	Aston Villa	63.2	57.0	6.2	57.2
9	Crystal Palace	55.3	60.8	-5.4	50.2
10	Brighton	54.0	61.5	-7.5	49.0
11	West Ham	55.2	63.4	-8.3	48.7
12	Bournemouth	51.0	64.4	-13.5	45.6
13	Brentford	49.6	65.1	-15.5	44.3
14	Everton	49.3	66.6	-17.3	43.5
15	Nott'ham Forest	48.9	66.3	-17.3	43.2
16	Fulham	48.3	67.2	-18.9	42.1
17	Wolves	48.0	68.1	-20.1	41.7
18	Leicester City	42.8	73.7	-30.8	35.6
19	Southampton	42.7	75.5	-32.8	34.8
20	Ipswich Town	40.6	77.2	-36.6	32.7

Figure 5.1: Average Forecast Table for 24/25 Premier League

therefore each of these squares relates to that team finishing in that position at least 100 times, i.e. Manchester United won the title in 1% (100) of the simulations. This effectively shows how much the simulations differ, which was to be expected when running them 'hot'. Manchester City won the title in nearly half (49.5%) of the simulations but also finished 14th in one of them, showing they went on a bad run of form which carried on for the rest of the season. The probabilities were also interesting at the other end, e.g even though Ipswich on average finish bottom of the league they actually only got relegated 57% of the time, i.e. finishing 18th, 19th or 20th.



	POS	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Manchester City	1	49.5	27.6	13.0	5.2	2.6	1.2														
Arsenal	2	33.8	32.8	16.5	8.7	4.2	2.1	1.0													
Liverpool	3	7.6	15.3	22.3	19.2	12.4	8.3	5.5	3.8	1.8	1.4										
Chelsea	4	6.5	14.1	22.1	20.2	13.5	9.4	5.2	3.3	2.0	1.5										
Manchester Utd	5	1.0	3.1	7.6	11.8	14.9	13.8	11.5	9.0	7.7	5.3	4.0	3.4	2.4	1.4	1.1					
Tottenham	6		3.0	6.5	11.2	13.1	13.2	11.5	9.7	7.7	6.0	4.5	3.1	2.6	2.5	1.6	1.1				
Newcastle Utd	7		1.8	4.8	8.2	12.1	11.9	12.1	10.5	8.8	7.5	6.1	4.5	3.6	2.5	2.0	1.5				
Aston Villa	8		1.4	3.5	6.9	10.2	11.3	11.7	11.1	9.6	8.1	6.4	4.8	3.7	3.3	2.7	2.2	1.1	1.0		
Crystal Palace	9			1.0	1.8	4.1	5.2	7.8	8.4	9.3	9.2	8.9	8.7	8.1	6.3	5.6	4.8	4.3	3.2	1.7	1.2
Brighton	10				1.5	2.9	5.1	6.4	8.2	9.2	8.7	8.4	8.3	8.5	7.6	6.1	5.6	4.3	4.1	2.4	1.6
West Ham	11				1.7	2.9	5.1	6.7	7.6	8.1	8.7	8.0	8.8	8.4	6.9	7.4	5.8	4.8	3.8	2.9	1.5
Bournemouth	12				1.0	1.7	3.1	4.7	5.3	6.0	7.4	8.3	8.4	8.7	8.4	9.1	7.3	6.6	6.1	4.7	2.8
Brentford	13					1.5	2.8	3.7	5.3	5.5	7.5	7.6	7.9	8.5	8.5	8.1	8.5	7.5	6.9	5.5	3.6
Everton	14					1.2	2.0	2.9	4.7	5.6	6.5	8.0	8.6	8.8	8.9	7.9	8.6	8.1	7.4	5.9	4.0
Nott'ham Forest	15						2.2	2.9	4.2	5.2	6.3	7.8	7.9	8.2	8.6	9.4	8.4	8.7	7.7	6.5	4.8
Fulham	16						1.5	2.3	3.4	4.6	5.5	6.4	7.0	7.0	9.7	8.9	9.1	9.4	9.2	8.3	6.3
Wolves	17						1.2	2.1	2.9	4.2	5.1	6.2	7.3	8.0	8.2	8.7	9.6	9.8	10.0	9.1	6.7
Leicester City	18									1.6	2.1	3.0	4.1	5.2	6.6	8.0	9.7	10.8	13.2	15.9	17.7
Southampton	19									1.6	1.7	2.4	3.3	4.0	5.4	7.4	9.2	11.6	12.8	17.2	21.9
Ipswich Town	20									1.0	1.3	2.1	2.5	3.6	4.9	5.7	7.7	10.5	13.3	18.7	27.4

Figure 5.2: Forecast Position Probabilities for 24/25 Premier League

Chapter 6

Conclusion

In this paper, we fitted a Poisson regression model and a logistic regression model to estimate goals scored by the home and away team in Premier League matches. We tested the models on the 23/24 season and simulated results 10,000 times to approximate their league table predictions. Both models performed with some success, predicting the correct winner and relegated teams. After comparing results and how the models scored in repeated k-fold validation, the Poisson model was deemed the most effective. We then used this model to forecast the 24/25 season, the average point predictions for each team along with the probability of finishing in each position as per our simulations, are shown in section 5.2. The results obtained by our model suggested this may be an effective method to have some success in predicting football matches and positions. However, without knowing the outcome of the 24/25 season we are unable to evaluate the models performance on out of sample data. All computations and graphics were performed using Python.

Whilst this paper explores some complex quantitative data in football, like expected goals and calculating rolling form averages, there are many other factors that can effect the score of a football match. For example our model does not account for injuries, which, to key players can significantly reduce the likelihood of a team winning. However, over a season one would hope that this evens out across all teams. Although our feature selection improves many other papers that focus on simple attack and defence ratings, the form variables could be improved further. For example a decaying weighting system could be applied to add a greater weight to recent matches which was

showcased effectively by Willis (2022) [32].

Limitations within the Poisson model revolve around the assumptions we had to make. The weakest assumption was that the mean and variance was equal which we found to be false with variance being greater than the mean. Therefore, in this context, perhaps a different distribution may be more appropriate, like the negative binomial GLM model which handles over dispersion by allowing the mean and variance to differ.

Both models failed to capture any dependency between home and away goals which we have assumed is minimal in this paper. However, if we wished to further develop this research, we could look into the use of models that account for this dependency. The classification task could be extended by fitting a classifier chain which uses the prediction of one target as an extra input variable for the prediction of the other target variables. Whilst this wouldn't account for total dependency as the first classifier will not know the result of the second, it would be a good next step. Other approaches could include the use of a random forest algorithms which Hucaljuk and Rakipovic (2011) used to classify games either as home win, away win or draw with relative success [9]. Therefore, another paper could explore adapting this approach to a goals based model with the more complex covariates used in this paper.

Appendix A

Source Code

A.1 Implementation of `def scrape_matches`

```
import time
import requests
from bs4 import BeautifulSoup
import pandas as pd

def scrape_matches(url, years, league):
    all_matches = []
    for year in years:
        data = requests.get(url)
        soup = BeautifulSoup(data.text)
        standings_table = soup.select('table.stats_table')[0] # find the
    league table

        links = [l.get("href") for l in standings_table.find_all('a')] #
    find the links to each team's match history
        links = [l for l in links if '/squads/' in l]
        team_urls = [f"https://fbref.com{l}" for l in links]

        previous_season = soup.select("a.prev")[0].get("href")
        url = f"https://fbref.com{previous_season}"

        for team_url in team_urls:
        # putting each team's match history for this season in a dataframe
            team_name = team_url.split("/")[-1].replace("-Stats", "").
    replace("-", " ")
            data = requests.get(team_url)
            matches = pd.read_html(data.text, match="Scores & Fixtures")[0]
            team_data = matches[matches["Comp"] == league]
            team_data["Season"] = year
            team_data["Team"] = team_name
            all_matches.append(team_data)
            time.sleep(7) # to stop getting blocked from url
```

```
    df = pd.concat(all_matches)
    df.columns = [c.lower() for c in df.columns]
    return df

champ_url = "https://fbref.com/en/comps/10/Championship-Stats"
prem_url = "https://fbref.com/en/comps/9/Premier-League-Stats"
years = list(range(2023, 2016, -1))
champ_matches = scrape_matches(champ_url, years, "Championship")
prem_matches = scrape_matches(prem_url, years, "Premier League")
```

A.2 Implementation of def get_market_value

```
def get_market_value(years, league, div, country):
    market_value_df = pd.DataFrame()
    for season in years:
        url = f'https://www.transfermarkt.co.uk/{league.lower()}/startseite/wettbewerb/{country}{div}/plus/?saison_id={season}' # URL pattern for each league and season
        headers = {
            "User-Agent":"Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/121.0.0.0 Safari/537.36"
            }
        response = requests.get(url, headers=headers)
        soup = BeautifulSoup(response.content, 'html.parser')
        teams = []
        list = soup.select('td[class="hauptlink no-border-links"]') # Selecting the team names
        for y in range(len(list)):
            team = list[y].text.strip()
            teams.append(team)
        df = pd.DataFrame(teams)
        df.columns = ['Team']
        df['Season'] = season
        mvalue = []
        value = soup.select('td[class="rechts"]') # Selecting the market values
        value = value[1::2]
        value = value[1:]
        for x in range(len(value)):
            mvalue.append(value[x].text.strip()) # Appending the market values to the list
        df['Market Value'] = mvalue
        df['Team'] = df['Team'].str.replace('FC','')
        df['Team'] = df['Team'].str.replace('A ','')
        df['Team'] = df['Team'].str.replace('&','and')
        df['Team'] = df['Team'].map(mapping)
        df['Team'] = df['Team'].str.strip()
        df['Market Value'] = df['Market Value'].str.replace('.','')
        df_temp = df['Market Value'].str.strip('€').str.extract(r'(\d+)([bnm]+)')
        df_temp2 = df_temp[0] + df_temp[1].map({'bn': '0000000', 'm':'0000'}) # Converting the market values to a common unit
        df.drop('Market Value', axis=1, inplace=True)
        df['Market Value'] = df_temp2
```

```
        market_value_df = pd.concat([market_value_df, df])
    return market_value_df

years = list(range(2023, 2016, -1))
league = 'premier-league'
div = '1'
country = 'GB'
market_value_df = get_market_value(years, league, div, country)
```

A.3 Implementation of def get_weighting

```
def get_weighting(metric):
    champ_prom_avg = champ_df[f"{metric}"].mean()
    prem_rel_avg = prem_df[f"{metric}"].mean()
    prem_rel_per_game = prem_rel_avg / 38
    champ_prom_per_game = champ_prom_avg / 46
    weighting = prem_rel_per_game - champ_prom_per_game
    return weighting
```

A.4 Implementation of def rolling averages for General Form Variables

```
# Function to take the rolling average of last n number of games
def rolling_averages(group, number, cols, new_cols):
    group = group.sort_values("date")
    rolling_stats = group[cols].rolling(number, closed='left').mean()
    group[new_cols] = rolling_stats
    return group

numbers = [5] # last 5 games
for number in numbers:
    cols = ["Adjusted GD"]
    new_cols = [f"{col} form last {number}" for col in cols]
    df = df.groupby("new_team").apply(rolling_averages, number, cols,
    new_cols)
    df = df.droplevel('new_team')
```

A.5 Implementation of def rolling averages for Home and Away form Variables

```
# Using rolling average function shown above
number = 10 # last 10 games
away_cols = ['away adjusted GS', 'away adjusted GC']
away_cols_new = [f"away_team_{c}_rolling_{number}" for c in away_cols]
matches = matches.groupby("away_team").apply(lambda x: rolling_averages(x,
    number,away_cols, away_cols_new))
```

```
6
7 home_cols = ['home adjusted GS', 'home adjusted GC']
8 home_cols_new = [f"home_team_{c}_rolling_{number}" for c in home_cols]
9 matches = matches.groupby("home_team").apply(lambda x: rolling_averages(x,
      number,home_cols, home_cols_new))
10
11 matches = matches.droplevel('away_team')
12 matches = matches.droplevel('home_team')
```

A.6 Implementation of def calculate_goal_probabilities

```
1 def calculate_goal_probabilities(lambda_param, max_goals=8):
2     probabilities = [poisson.pmf(i, lambda_param) for i in range(max_goals)]
3     probabilities.append(1 - sum(probabilities))  # Probability of max_goals
       or more
4     return probabilities
```

A.7 Implementation of def simulate_game

```
1 def simulate_home_goals(home_goal_probs):
2     result = np.random.choice([0,1,2,3,4,5,6,7,8], p=home_goal_probs)
3     return result
4
5 def simulate_away_goals(away_goal_probs):
6     result = np.random.choice([0,1,2,3,4,5,6,7,8], p=away_goal_probs)
7     return result
8
9 def simulate_game(game, home_goal_probs, away_goal_probs):
10    home_goals = simulate_home_goals(home_goal_probs.iloc[game])
11    away_goals = simulate_away_goals(away_goal_probs.iloc[game])
12    return home_goals, away_goals
```

A.8 Implementation of def simulate_all_games

```
1 def simulate_all_games(match_df, home_goal_df,away_goal_df, num_iterations):
2     teams = match_df['HT'].unique()
3     # Set counters for total simulations
4     team_positions = {team: Counter() for team in teams}
5     total_points = {team: 0 for team in teams}
6     total_goals_scored = {team: 0 for team in teams}
7     total_goals_conceded = {team: 0 for team in teams}
8     for _ in range(num_iterations):
9         # Set counters for each simulated season
10        team_points = {team: 0 for team in teams}
11        team_goals_scored = {team: 0 for team in teams}
12        team_goals_conceded = {team: 0 for team in teams}
13
14        for game in range(len(match_df)):
```

```
15              home_team = match_df.loc[game, 'HT']
16              away_team = match_df.loc[game, 'AT']
17              home_goals, away_goals = simulate_game(game, home_goal_df,
    away_goal_df)
18              if home_goals > away_goals: # Home Win
19                  team_points[home_team] += 3
20              elif home_goals == away_goals: # Draw
21                  team_points[home_team] += 1
22                  team_points[away_team] += 1
23              else:  # Away Win
24                  team_points[away_team] += 3
25              team_goals_scored[home_team] += home_goals
26              team_goals_scored[away_team] += away_goals
27              team_goals_conceded[home_team] += away_goals
28              team_goals_conceded[away_team] += home_goals
29
30          # Sort teams based on points and record positions
31          total_points = {team: total_points[team] + team_points[team] for
    team in teams}
32          total_goals_scored = {team: total_goals_scored[team] +
    team_goals_scored[team] for team in teams}
33          total_goals_conceded = {team: total_goals_conceded[team] +
    team_goals_conceded[team] for team in teams}
34          sorted_teams = sorted(team_points.items(), key=lambda x: x[1],
    reverse=True)
35          for position, (team, _) in enumerate(sorted_teams, start=1):
36              team_positions[team][position] += 1
37
38      sorted_table = sorted(total_points.items(), key=lambda x: x[1], reverse=
    True)
39      # Find the Averages of each metric
40      average_goals_scored = pd.DataFrame(total_goals_scored.items(), columns
    =['Team', 'Goals Scored'])
41      average_goals_conceded = pd.DataFrame(total_goals_conceded.items(),
    columns=['Team', 'Goals Conceded'])
42      average_points = pd.DataFrame(sorted_table, columns=['Team', 'Points'])
43      average_points = average_points.merge(average_goals_scored, on='Team',
    how='left')
44      average_points = average_points.merge(average_goals_conceded, on='Team',
     how='left')
45      average_points['Points'] = average_points['Points'] / num_iterations
46      average_points['Goals Scored'] = average_points['Goals Scored'] /
    num_iterations
47      average_points['Goals Conceded'] = average_points['Goals Conceded'] /
    num_iterations
48      average_points['Goal Difference'] = average_points['Goals Scored'] -
    average_points['Goals Conceded']
49      average_points.loc[average_points['Team'] == 'Everton', 'Points'] -= 6
50      average_points.loc[average_points['Team'] == 'Nott\'ham Forest', 'Points
    '] -= 4
51      average_points.sort_values(by='Points', ascending=False, inplace=True)
52      average_points['Position'] = average_points.index + 1
53      average_points_table = average_points[['Position','Team', 'Points', '
    Goals Scored', 'Goals Conceded', 'Goal Difference']]
54      return team_positions, average_points_table
```

A.9 Implementation of def simulate_all_future_games

```
def simulate_all_future_games(forecast_df, df_future, home_goal_model, away_goal_model, home_predictors, away_predictors, num_iterations, home_trainX, away_trainX):
    teams = df_future['HT'].unique()
    # Set counters for total simulations
    team_positions = {team: Counter() for team in teams}
    total_points = {team: 0 for team in teams}
    total_goals_scored = {team: 0 for team in teams}
    total_goals_conceded = {team: 0 for team in teams}
    # Scale Data
    home_scaler = StandardScaler()
    home_scaler.fit(home_trainX)
    away_scaler = StandardScaler()
    away_scaler.fit(away_trainX)

    for _ in range(num_iterations):
        # Set counters for each simulated season
        team_points = {team: 0 for team in teams}
        team_goals_scored = {team: 0 for team in teams}
        team_goals_conceded = {team: 0 for team in teams}

        for gameweek in range(38):
            # Each Season
            df_test = df_future.copy() # Retrieve the updated dataframe
            df_test.reset_index(drop=True, inplace=True)
            home_df_testX = df_test[home_predictors] # Using home model predictors
            away_df_testX = df_test[away_predictors] # Using away model predictors
            home_df_testX_scaled = home_scaler.transform(home_df_testX, copy=True)
            away_df_testX_scaled = away_scaler.transform(away_df_testX, copy=True)
            pred_home_goals = home_goal_model.predict(home_df_testX_scaled[gameweek*10:gameweek*10+10]) * 1.0435 # Adjusted for increased match time
            pred_away_goals = away_goal_model.predict(away_df_testX_scaled[gameweek*10:gameweek*10+10]) * 1.0435
            future_home_goals_perc = pd.DataFrame([calculate_goal_probabilities(goals) for goals in pred_home_goals]) # Find goal probabilities from Poisson PMF
            future_away_goals_perc = pd.DataFrame([calculate_goal_probabilities(goals) for goals in pred_away_goals])

            for game in range(10):
                # Simulate each game in the Gameweek
                home_points, away_points, home_goals, away_goals = simulate_game_future(future_home_goals_perc.iloc[game], future_away_goals_perc.iloc[game])
                actual_game = game + gameweek*10
                team_points[df_test.iloc[actual_game]['HT']] += home_points
                team_points[df_test.iloc[actual_game]['AT']] += away_points
                team_goals_scored[df_test.iloc[actual_game]['HT']] += home_goals
                team_goals_scored[df_test.iloc[actual_game]['AT']] +=
```

```
    away_goals
41                  team_goals_conceded[df_test.iloc[actual_game]['HT']] +=
    away_goals
42                  team_goals_conceded[df_test.iloc[actual_game]['AT']] +=
    home_goals
43                  # Append Result to Dataframe
44                  forecast_df = append_result(forecast_df, df_test,
    actual_game, home_points, away_points, home_goals, away_goals)
45
46              # Update Form based on these results
47              df_future = update_form(forecast_df)
48
49          # Calculate end of season team totals
50          total_points = {team: total_points[team] + team_points[team] for
    team in teams}
51          total_goals_scored = {team: total_goals_scored[team] +
    team_goals_scored[team] for team in teams}
52          total_goals_conceded = {team: total_goals_conceded[team] +
    team_goals_conceded[team] for team in teams}
53          sorted_teams = sorted(team_points.items(), key=lambda x: x[1],
    reverse=True)
54
55          # Keeping track of number of times each team finished in each
    position
56          for position, (team, _) in enumerate(sorted_teams, start=1):
57              team_positions[team][position] += 1
58
59      # Working out average table from all simulations
60      sorted_table = sorted(total_points.items(), key=lambda x: x[1], reverse=
    True)
61      average_goals_scored = pd.DataFrame(total_goals_scored.items(), columns
    =['Team', 'Goals Scored'])
62      average_goals_conceded = pd.DataFrame(total_goals_conceded.items(),
    columns=['Team', 'Goals Conceded'])
63      average_points = pd.DataFrame(sorted_table, columns=['Team', 'Points'])
64      average_points = average_points.merge(average_goals_scored, on='Team',
    how='left')
65      average_points = average_points.merge(average_goals_conceded, on='Team',
     how='left')
66      average_points['Points'] = average_points['Points'] / num_iterations
67      average_points['Goals Scored'] = average_points['Goals Scored'] /
    num_iterations
68      average_points['Goals Conceded'] = average_points['Goals Conceded'] /
    num_iterations
69      average_points['Goal Difference'] = average_points['Goals Scored'] -
    average_points['Goals Conceded']
70      average_points['Position'] = average_points.index + 1
71      average_points_table = average_points[['Position','Team', 'Points', '
    Goals Scored', 'Goals Conceded', 'Goal Difference']]
72
73      return team_positions, average_points_table
```

A.10 Implementation of `def bootstrap`

```
from sklearn.utils import resample
from sklearn.model_selection import train_test_split

def bootstrap(df, M, train, test, target, predictors, alpha):
    features_list = []
    for bootstrap in range(M): # number of re-samples
        bootstrap_dataset = resample(df, n_samples=len(df))
        train, test = train_test_split(bootstrap_dataset, test_size=0.2) # Split Data into 80% training, 20% test
        trainX, trainY, testX, testY = get_feature_split(train, test, target, predictors)
        trainX_scaled, testX_scaled = get_scaled_data(trainX, testX)
        goal_model = fit_poisson_model(alpha, True, 100, trainX_scaled, trainY)
        features_list.append(goal_model.coef_) # Append model coefficients to list

    flattened_boostrap_weights = [inner_array.ravel() for inner_array in features_list]
    return flattened_boostrap_weights
```

A.11 Implementation of `def grid_search`

```
from sklearn.linear_model import PoissonRegressor
from sklearn.model_selection import GridSearchCV

def grid_search(trainX, trainY):
# select the params to search through
    param_grid = {
        'alpha': np.arange(0, 1, 0.1),
        'max_iter': [100, 200, 500],
        'fit_intercept': [True, False]
    }
    pr = PoissonRegressor()
    gs = GridSearchCV(pr, param_grid, cv=5, scoring='neg_mean_squared_error') # use MSE to choose optimal params
    model = gs.fit(trainX, trainY) #fit model using optimal params
    return model
home_goal_model = grid_search(home_trainX_scaled, home_trainY)
away_goal_model = grid_search(away_trainX_scaled, away_trainY)
```

A.12 Implementation of `def grid_search_log_reg`

```
def grid_search_log_reg(trainX_scaled, trainY, testX_scaled, testY, home_goals, away_goals):
    home_goals = np.arange(0, 10)
    away_goals = np.arange(0, 10)
    C = np.arange(0.1, 1.1, 0.1)
    max_iter = [200,500,1000]
    intercept = [True]
    mse_values = []
```

```
    c_values = []
    iter_values = []
    intercept_values = []
    for i in C:
        for j in max_iter:
            for k in intercept:
                goal_model = fit_log_reg_model(i, j, k, trainX_scaled, trainY)
                home_goal_probs, away_goal_probs = get_goal_prediction_perc(goal_model, testX_scaled)
                home_goal_probs = shift_probabilities(home_goal_probs)
                away_goal_probs = shift_probabilities(away_goal_probs)
                mean_home_goals = []
                for x in range(len(home_goal_probs)):
                    mean_home_goals.append(np.sum(home_goals * home_goal_probs.iloc[x]))
                mean_away_goals = []
                for y in range(len(away_goal_probs)):
                    mean_away_goals.append(np.sum(away_goals * away_goal_probs.iloc[y]))
                mse = (np.mean((mean_home_goals - testY['home GF'])**2) + np.mean((mean_away_goals - testY['away GF'])**2))/2
                mse_values.append(mse)
                c_values.append(i)
                iter_values.append(j)
                intercept_values.append(k)

    return pd.DataFrame(list(zip(c_values, iter_values, intercept_values, mse_values)), columns=['c_values', 'max_iter', 'intercept', 'MSE'])
```

A.13 Implementation of `def kfold_cv`

```
from sklearn.model_selection import KFold
def kfold_cv(trainX_scaled, trainY, k, n):
    scores = []
    for i in range(n): # number of repetitions
        kfold = KFold(n_splits=k, shuffle=True)
        for train_index, test_index in kfold.split(trainX_scaled):
            X_train, X_test = trainX_scaled[train_index], trainX_scaled[test_index]
            y_train, y_test = trainY.iloc[train_index], trainY.iloc[test_index]
            home_goal_model = fit_poisson_model(0.2, True, 100, X_train, y_train['home GF'])
            away_goal_model = fit_poisson_model(0.2, True, 100, X_train, y_train['away GF'])
            pred_home_goals = home_goal_model.predict(X_test)
            pred_away_goals = away_goal_model.predict(X_test)
            mse = (np.mean((pred_home_goals - y_test['home GF'])**2) + np.mean((pred_away_goals - y_test['away GF'])**2))/2 # Averaging MSE of both models
            scores.append(mse)
            score = np.mean(scores) # Mean score over all repetitions
    return score
```

Appendix B

References

Bibliography

[1] Andrews, Z. (2019), 'Comparing Predictive Models for English Premier League Games'. Available at: https://libres.uncg.edu/ir/asu/f/Andrews_Zachary_2019_Thesis.pdf (Accessed: 28 August 2024)

[2] Azhari, H. R. *et al.* (2018), 'Predicting Final Result of Football Match Using Poisson Regression Model', *Journal of Physics: Conference Series*, 1108(1). Available at: https://iopscience.iop.org/article/10.1088/1742-6596/1108/1/012066 (Accessed: 28 August 2024)

[3] Bruinsma, R. (2020), 'Using Poisson regression to model football scores and exploit inaccuracies in the online betting market'. Available at: https://fse.studenttheses.ub.rug.nl/21917/1/bMATH_2020_BruinsmaR.pdf (Accessed: 28 August 2024)

[4] Cambridge Dictionary (2024) *be in good, great, etc. form*. Available at: https://dictionary.cambridge.org/dictionary/english/be-in-good-great-etc-form (Accessed: 28 August 2024)

[5] Carron, A. V. *et al.* (2005), 'The home advantage in sport competitions: Courneya and Carron's (1992) conceptual framework a decade later', *Journal of Sports Sciences*, 23(4), pp. 395–407. Available at: https://www.tandfonline.com/doi/abs/10.1080/02640410400021542 (Accessed: 28 August 2024)

[6] Fbref (2024a) *2023-2024 Premier League Stats*. Available at: https://fbref.com/en/comps/9/2023-2024/2023-2024-Premier-League-Stats (Accessed: 28 August 2024)

[7] Fbref (2024b) *2023-2024 Manchester City Stats (Premier League).* Available at: https://fbref.com/en/squads/b8fd03ef/2023-2024/Manchester-City-Stats (Accessed: 28 August 2024)

[8] Fbref (2024c) *xG Explained*. Available at: https://fbref.com/en/expected-goals-model-explained/(Accessed: 28 August 2024)

[9] Hucaljuk, J. and Rakipović, A. (2011) 'Predicting football scores using machine learning techniques'. Available at: https://ieeexplore.ieee.org/abstract/document/5967321 (Accessed: 28 August 2024)

[10] Karlis, D. and Ntzoufras, I. (2009) 'Bayesian modelling of football outcomes: using the Skellam's distribution for the goal difference', *IMA Journal of Management Mathematics*, 20, pp. 133-145. Available at: https://www.researchgate.net/publication/228621612_Bayesian_modelling_of_football_outcom (Accessed: 28 August 2024)

[11] Maozad, S. N. *et al.* (2022) 'Comparative analysis for predicting football match outcomes based on Poisson models'. Available at: https://ieeexplore.ieee.org/abstract/document/9795385 (Accessed: 28 August 2024)

[12] Minitab (2024) *What is a link function?*. Available at: https://support.minitab.com/en-us/minitab/help-and-how-to/statistical-modeling/regression/supporting-topics/logistic-regression/link-function/ (Accessed: 28 August 2024)

[13] Nguyen, Q. (2021) 'Poisson Modeling and Predicting English Premier League Goal Scoring', *The New England Journal of Statistics in Data Science*, 1(1), pp. 95-101. Available at: https://nejsds.nestat.org/journal/NEJSDS/article/1/info (Accessed: 28 August 2024)

[14] Opta Analyst (2024) *Premier League Predictions 2024-25: The Opta Projections*. Available at: https://theanalyst.com/eu/2024/08/premier-league-predictions-2024-25-opta-supercomputer (Accessed: 28 August 2024)

[15] Transfermarkt (2024a) *UEFA 5-Year Rankings*. Available at: https://www.transfermarkt.co.uk/uefa/5jahreswertung/statistik (Accessed: 28 August 2024)

[16] Transfermarkt (2024b) *Overall Market Value Change in all Clubs in the Premier League*. Available at: https://www.transfermarkt.co.uk/premier-league/startseite/wettbewerb/GB1 (Accessed: 28 August 2024)

[17] Saraiva, E. F. *et al.* (2016) 'Predicting football scores via Poisson regression model: applications to the National Football League', *Communications for Statistical Applications and Methods*, 23, pp. 297-319. Available at: http://www.csam.or.kr/journal/view.html?doi=10.5351/CSAM.2016.23.4.297 (Accessed: 28 August 2024)

[18] Scikit Learn (2024a) *GridSearchCV*. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html (Accessed: 28 August 2024)

[19] Scikit Learn (2024b) *Cross-validation: evaluating estimator performance*. Available at: https://scikit-learn.org/stable/modules/cross_validation.html (Accessed: 28 August 2024)

[20] Scikit Learn (2024c) *PoissonRegressor*. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.PoissonRegressor.html (Accessed: 28 August 2024)

[21] Scikit Learn (2024d) *Multiclass and multioutput algorithms*. Available at: https://scikit-learn.org/stable/modules/multiclass.html (Accessed: 28 August 2024)

[22] Scikit Learn (2024e) *LogisticRegression*. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html (Accessed: 28 August 2024)

[23] Statista (2023) *Average amount of in-play time in Premier League games from 2012/13 to 2023/[24*. Available at: https://www.statista.com/statistics/1359233/premier-league-in-play-time/ (Accessed: 28 August 2024)

[24] Wikipedia (2024a) *Logistic Regression*. Available at: https://en.wikipedia.org/wiki/Logistic_regression (Accessed: 28 August 2024)

[25] Wikipedia (2024b) *Multinomial Logistic Regression*. Available at: https://en.wikipedia.org/wiki/Multinomial_logistic_regression (Accessed: 28 August 2024)

[26] Wikipedia (2024c) *Maximum Likelihood Estimation*. Available at: https://en.wikipedia.org/wiki/Maximum_likelihood_estimation (Accessed: 28 August 2024)

[27] Wikipedia (2024d) *Independence (probability theory)*. Available at: https://en.wikipedia.org/wiki/Independence_(probability_theory) (Accessed: 28 August 2024)

[28] Wikipedia (2024e) *Poisson Distribution*. Available at: https://en.wikipedia.org/wiki/Poisson_distribution (Accessed: 28 August 2024)

[29] Wikipedia (2024f) *Softmax Function*. Available at: https://en.wikipedia.org/wiki/Softmax_function (Accessed: 28 August 2024)

[30] Wikipedia (2024g) *Sigmoid Function*. Available at: https://en.wikipedia.org/wiki/Sigmoid_function (Accessed: 28 August 2024)

[31] Wikipedia (2024h) *Poisson Regression*. Available at: https://en.wikipedia.org/wiki/Poisson_regression (Accessed: 28 August 2024)

[32] Willis, S. (2022) *Explaning my Simulation Model*. Available at: https://www.cannonstats.com/p/explaining-my-simulation-model (Accessed: 28 August 2024)

[33] Zebari, G, M. *et al*. (2021) 'Predicting Football Outcomes by Using Poisson Model: Applied to Spanish Primera División', *Journal of Applied Science and Technology Trends*, 2(02), pp. 133–140. Available at: https://jastt.org/index.php/jasttpath/article/view/112 (Accessed: 28 August 2024)